LUCRĂRI DE LABORATOR DE CALCUL NUMERIC

ALEXANDRU MIHAI BICA, EUGEN LASLO,
MUSCAŞ ANNA MONIKA, PELLE REMUS NICOLAE

UNIVERSITATEA din ORADEA

Cuprins

Introducere **5**

1 Laboratorul 1. Sisteme liniare. Metode directe **7**

2 Laboratorul 2. Sisteme liniare. Metode iterative **15**

3 Laboratorul 3. Ecuaţii neliniare. Metoda coardei **21**

4 Laboratorul 4. Metoda tangentei **27**

5 Laboratorul 5. Metoda aproximaţiilor succesive **33**

6 Laboratorul 6. Sisteme neliniare. Metode iterative **39**

7 Laboratorul 7. Sisteme neliniare. Metoda lui Newton **45**

8 Laboratorul 8. Polinomul Taylor de aproximare locală **51**

9 Laboratorul 9. Polinomul de interpolare Lagrange **59**

10 Laboratorul 10. Polinoame de interpolare Hermite şi polinomul Bernstein **65**

11 Laboratorul 11. Regresia liniară, pătratică şi funcţii spline liniare **73**

12 Laboratorul 12. Funcţii spline cubice **81**

13 Laboratorul 13. Cuadraturi de tip trapezoidal **85**

14 Laboratorul 14. Cuadraturi de tip Simpson şi Newton **93**

15 Bibliografie **99**

Introducere

Prezentul îndrumător de lucrări de laborator de calcul numeric este conceput în legatură cu cursul de calcul numeric destinat studenților de anul III ai specializării Informatică și este organizat pe 14 capitole corespunzând celor 14 săptămâni semestriale. Este format din patru părți: rezolvarea sistemelor liniare (laboratoarele 1 și 2), rezolvarea numerică a ecuațiilor și sistemelor algebrice neliniare (laboratoarele 3, 4, 5, 6 și 7), aproximarea polinomială și spline a funcțiilor (laboratoarele 8, 9, 10, 11 și 12), și integrarea numerică (laboratoarele 13 și 14). Laboratoarele 8-12 implică și reprezentarea grafică a funcției de interpolare și aproximare, fiind necesare și cunoștințe de proiectare grafică.

Capitolul 1

Laboratorul 1. Sisteme liniare. Metode directe

REZOLVAREA SISTEMELOR LINIARE. METODE DIRECTE (exacte)

Considerăm sistemul liniar

$$\begin{cases} a_{11} \cdot x_1 + ... + a_{1n} \cdot x_n = b_1 \\ ..... \\ a_{n1} \cdot x_1 + ... + a_{nn} \cdot x_n = b_n \end{cases}$$

ce poate fi scris sub forma vectorială $A \cdot x = b$, unde $A = \begin{pmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ ... & ... & ... & ... \\ a_{n1} & a_{n2} & ... & a_{nn} \end{pmatrix}$, $x = \begin{pmatrix} x_1 \\ ... \\ x_n \end{pmatrix}$ şi $b = \begin{pmatrix} b_1 \\ ... \\ b_n \end{pmatrix}$. Presupunem că $\det A \neq 0$.

1) **Sisteme superior triunghiulare**

În cazul sistemelor superior triunghiulare matricea A are forma

$$A = \begin{pmatrix} a_{11} & a_{12} & a_{13} & ... & a_{1n} \\ 0 & a_{22} & a_{23} & ... & a_{2n} \\ 0 & 0 & a_{33} & ... & a_{3n} \\ ... & ... & ... & \ddots & ... \\ 0 & 0 & 0 & ... & a_{nn} \end{pmatrix}$$

cu toate elementele nule sub diagonala principală. În acest caz condiția $\det A \neq 0$ este echivalentă cu a cere $a_{ii} \neq 0$, $\forall i = \overline{1, n}$.

Algoritmul "Sistem superior triunghiular"

I. Date de intrare:

n (int)

$a[i, j]$, $i, j = \overline{1, n}$, astfel încât $a[i, j] = 0$ pentru $i > j$ (double)

b_i, $i = \overline{1,n}$ (double)
II. Ieșire: $x[k]$, $k = \overline{1,n}$
III. Pași:
1. Execută $x[n] := b[n]/a[n,n]$
2. Pentru $k = \overline{n-1,1}$ execută
   $S := 0$
      Pentru $i = \overline{k+1,n}$ execută $S := S + a[k,i] \cdot x[i]$
   $x[k] := (b[k] - S)/a[k,k]$
3. Pentru $k = \overline{1,n}$ tipărește $x[k]$. Stop.

Exemplu: $A = \begin{pmatrix} 1 & 2 & 3 \\ 0 & -2 & 4 \\ 0 & 0 & 6 \end{pmatrix}$, $b = \begin{pmatrix} 5 \\ 2 \\ 3 \end{pmatrix}$

2) **Sisteme inferior triunghiulare**

În cazul sistemelor inferior triunghiulare matricea A are forma

$$A = \begin{pmatrix} a_{11} & 0 & 0 & ... & 0 \\ a_{21} & a_{22} & 0 & ... & 0 \\ a_{31} & a_{32} & a_{33} & ... & 0 \\ ... & ... & ... & \ddots & ... \\ a_{n1} & a_{n2} & a_{n3} & ... & a_{nn} \end{pmatrix}$$

cu toate elementele nule deasupra diagonalei principale. În acest caz condiția $\det A \neq 0$ este echivalentă cu a cere $a_{ii} \neq 0$, $\forall i = \overline{1,n}$.

Algoritm
I. Date de intrare:
n (int)
$a[i,j]$, $i,j = \overline{1,n}$, astfel încat $a[i,j] = 0$ pentru $i < j$ (double)
b_i, $i = \overline{1,n}$ (double)
II. Ieșire: $x[k]$, $k = \overline{1,n}$
III. Pași:
1. Execută $x[1] := b[1]/a[1,1]$
2. Pentru $k = \overline{2,n}$ execută
   $S := 0$
      Pentru $i = \overline{1,k-1}$ execută $S := S + a[k,i] \cdot x[i]$
   $x[k] := (b[k] - S)/a[k,k]$
3. Pentru $k = \overline{1,n}$ tipărește $x[k]$. Stop.

Exemplu: $A = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 4 & -1 & 6 \end{pmatrix}$, $b = \begin{pmatrix} 2 \\ 4 \\ 1 \end{pmatrix}$

3) **Metoda eliminării a lui Gauss**

Se aplică atunci când reușim să transformăm sistemul astfel încat toate elementele de pe diagonala principală a matricii sistemului să fie nenule.

Algoritm

I. Date de intrare:
n (int)
$a[i,j]$, $i,j=\overline{1,n}$, (double)
b_i, $i=\overline{1,n}$ (double)
II. Date de ieșire: $x[k]$, $k=\overline{1,n}$
III. Pași:
1. Pentru $k=\overline{1,n-1}$ execută
dacă $a[k,k]\neq 0$ atunci $p:=a[k,k]$, altfel Stop;
Pentru $j=\overline{k,n}$ execută $a[k,j]:=\frac{a[k,j]}{p}$
$b[k]:=\frac{b[k]}{p}$
Pentru $i=\overline{k+1,n}$ execută
Pentru $j=\overline{k+1,n}$ execută $a[i,j]:=a[i,j]-a[k,j]\cdot a[i,k]$
$b[i]:=b[i]-b[k]\cdot a[i,k]$
2. Aplică algoritmul "Sistem superior triunghiular"
3. Pentru $k=\overline{1,n}$ tipărește $x[k]$.
Stop.

}*Exemple numerice*:
1. Sistemul prezentat în cadrul implementării algoritmului de mai sus este

$$\begin{cases} 10x_1+2x_2-x_3=9 \\ -3x_1+x_2-5x_3=-8 \\ -2x_1-5x_2+x_3=-1 \end{cases}$$

pentru verificare soluția este: $x_1=1$, $x_2=0$, $x_3=1$

4) **Rezolvarea sistemelor tridiagonale**

În cazul sistemelor tridiagonale matricea sistemului are elemente nenule doar pe diagonala principală și pe cele două "mini-diagonale" situate una imediat sub diagonală, iar cealaltă imediat deasupra diagonalei. Celelalte elemente ale matricii sunt toate nule. Astfel matricea are forma

$$A=\begin{pmatrix} a_0 & c_0 & 0 & 0 & ... & ... & 0 \\ b_1 & a_1 & c_1 & 0 & ... & ... & 0 \\ ... & \ddots & \ddots & \ddots & ... & ... & .. \\ 0 & ...0 & b_i & a_i & c_i & 0... & 0 \\ ... & ... & ... & \ddots & \ddots & \ddots & ... \\ 0 & ... & ... & 0 & b_{n-1} & a_{n-1} & c_{n-1} \\ 0 & ... & ... & 0 & 0 & b_n & a_n \end{pmatrix}$$

iar un astfel de sistem este scris sub forma

$$\begin{cases} a_0x_0+c_0x_1=d_0 \\ ..... \\ b_ix_{i-1}+a_ix_i+c_ix_{i+1}=d_i, \quad i=\overline{1,n-1} \\ ..... \\ b_nx_{n-1}+a_nx_n=d_n \end{cases}$$

Se rezolvă sistemul prin metoda eliminării a lui Gauss care în acest caz generează un algoritm iterativ de obținere a soluției exacte.

Algoritm

I. Date de intrare:

a_i, d_i, $i = \overline{0,n}$; b_i, $i = \overline{1,n}$; c_i, $i = \overline{0,n-1}$ (double)

II. Ieșire: $x[k]$, $k = \overline{0,n}$

III. Pași:

1. Execută $u_0 := c_0/a_0$
2. Pentru $i = \overline{1,n-1}$ execută
   {
   $w_i := a_i - u_{i-1} \cdot b_i$
   $u_i := c_i/w_i$
   }
3. Execută $w_n := a_n - u_{n-1} \cdot b_n$
4. Execută $z_0 := d_0/a_0$
   Pentru $i = \overline{1,n}$ execută $z_i := (d_i - b_i \cdot z_{i-1})/w_i$
5. Execută $x_n := z_n$
   Pentru $i = \overline{n-1,0}$ execută $x_i := z_i - u_i \cdot x_{i+1}$
6. Pentru $k = \overline{0,n}$ tipărește x_k . Stop.

Exemplu:

Să se rezolve sistemul tridiagonal

$$\begin{cases} x_0 - \frac{1}{4}x_1 = -\frac{21}{2} \\ -\frac{1}{8}x_0 + x_1 - \frac{1}{8}x_2 = \frac{29}{4} \\ -\frac{1}{8}x_1 + x_2 - \frac{1}{8}x_3 = \frac{139}{16} \\ -\frac{1}{8}x_2 + x_3 - \frac{1}{8}x_4 = -\frac{51}{4} \\ -\frac{1}{8}x_3 + x_4 - \frac{1}{8}x_5 = \frac{157}{16} \\ -\frac{1}{4}x_4 + x_5 = -\frac{13}{2} \end{cases}$$

Pentru verificare, soluția cu aproximație la 4 cifre semnificative exacte este:

$$x_0 = -8.7018, \quad x_1 = 7.1929, \quad x_2 = 8.2452$$

$$x_3 = -10.731, \quad x_4 = 7.9057, \quad x_5 = -4.5236$$

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

În primul rând, observăm că toate procedeele prezentate mai sus au niște proprietăți comune, mai exact proprietățile care definesc un sistem: numărul de ecuații și de necunoscute ale sistemului n, matricea A a sistemului, vectorul b al sistemului și vectorul x de necunoscute al sistemului. Acest lucru ar presupune crearea unei *clase*, iar pentru că aceasta are rolul doar de a avea proprietățile comune într-un singur loc, nu dorim să o folosim în mod direct, deci această clasă va fi definită *abstractă*. Constructorul primește cele 3 date de intrare, n, A și b, iar pe x îl vom inițializa cu lungimea n. Vom mai avea încă o metodă, *Rezolvare*, unde vom avea implementări diferite pentru fiecare clasă derivată, deci și această metodă este *abstractă*.

Algorithm 1 Clasa de bază Sistem

```
1: public abstract class Sistem
2: {
3:     public int n;
4:     public decimal[,] A;
5:     public decimal[] b;
6:     public decimal[] x;
7:     public Sistem(int n, decimal[,] A, decimal[] b)
8:     {
9:         this.n = n;
10:        this.A = A;
11:        this.b = b;
12:        x = new decimal[n];
13:    }
14:    public abstract void Rezolvare();
15: }
```

Acum că avem această clasă de bază, pentru fiecare dintre procedeele prezentate, vom crea câte o clasă derivată. Pentru sistemele superior sau inferior triunghiulare, clasele vor fi foarte simple:

Algorithm 2 Clasa SistemSuperiorTriunghiular

```
1: public class SistemSuperiorTriunghiular :  Sistem
2: {
3:     public SistemSuperiorTriunghiular(int n, decimal[,] A,
4:         decimal[] b) :  base(n, A, b) { }
5:     public override void Rezolvare()
6:     {
7:         // Algoritm
8:     }
9: }
```

Pentru verificarea exemplelor, în metoda *Main*, inițializăm clasa folosind valorile definite în exemplu.

Algorithm 3 Folosirea claselor în metoda Main

```
1: static void Main(string[] args)
2: {
3:     int n = 3;
4:     decimal[,] A = new decimal[,]
5:     {
6:         { 1, 0, 0 },
7:         { 2, 3, 0 },
8:         { 4, -1, 6 }
9:     };
10:    decimal[] b = new decimal[] { 2, 4, 1 };
11:    Sistem infTriunghiular = new SistemInferiorTriunghiular(n, A, b);
12:    infTriunghiular.Rezolvare();
13:
14:    Console.WriteLine("Solutia sistemului este:");
15:    for(int i = 0; i < n; i++)
16:        Console.Write(infTriunghiular.x[i] + " ");
17: }
```

Pentru metoda eliminării a lui Gauss, observăm că parte din algoritm este aplicarea algoritmului pentru sistemul superior triunghiular. Pentru că avem deja acel algoritm scris, putem deriva clasa *MetodaEliminariiGauss* din clasa *SistemSuperiorTriunghiular*, iar la finalul metodei *Rezolvare*, putem aplica funcționalitatea de bază, care va accesa algoritmul rezolvării sistemului superior triunghiular automat.

Algorithm 4 Clasa MetodaEliminariiGauss

```
1: public class MetodaEliminariiGauss :  SistemSuperiorTriunghiular
2: {
3:     public MetodaEliminariiGauss(int n, decimal[,] A, decimal[] b)
4:         :  base(n, A, b) { }
5:     public override void Rezolvare()
6:     {
7:         // Algoritm
8:         base.Rezolvare();
9:     }
10: }
```

Deși sistemul tridiagonal are aceeași bază ca și celelalte sisteme, are și alte proprietăți specifice: vectorii *a*, *c* și *d* ai sistemului, iar vectorul *d* are scopul vectorului *b* dinainte. Vom declara și aceste proprietăți, pe care le primim și inițializăm în constructor. Observăm de asemenea, că *n* nu mai reprezintă numărul de soluții, în schimb avem *n+1* soluții, deci reinițializăm *x* în constructor cu dimensiunea potrivită.

Algorithm 5 Clasa SistemTridiagonal

```
1: public class SistemTridiagonal :  Sistem
2: {
3:     public decimal[] a;
4:     public decimal[] c;
5:     public decimal[] d;
6:     public SistemTridiagonal(int n, decimal[,] A, decimal[] a,
7:         decimal[] b, decimal[] c, decimal[] d) :  base(n, A, b)
8:     {
9:         this.a = a;
10:        this.c = c;
11:        this.d = d;
12:        x = new decimal[n + 1];
13:    }
14:    public override void Rezolvare()
15:    {
16:        // Algoritm
17:    }
18: }
```

Capitolul 2

Laboratorul 2. Sisteme liniare. Metode iterative

METODE ITERATIVE PENTRU REZOLVAREA SISTEMELOR LINIARE

Metoda lui Iacobi

Pentru rezolvarea sistemului

$$\begin{cases} a_{11} \cdot x_1 + ... + a_{1n} \cdot x_n = b_1 \\ ..... \\ a_{n1} \cdot x_1 + ... + a_{nn} \cdot x_n = b_n \end{cases}$$

scris sub foma vectorială $A \cdot x = b$, unde $A = \begin{pmatrix} a_{11} & a_{12} & ... & a_{1n} \\ a_{21} & a_{22} & ... & a_{2n} \\ ... & ... & ... & ... \\ a_{n1} & a_{n2} & ... & a_{nn} \end{pmatrix}$, $x = \begin{pmatrix} x_1 \\ ... \\ x_n \end{pmatrix}$ şi $b = \begin{pmatrix} b_1 \\ ... \\ b_n \end{pmatrix}$, dacă $a_{ii} \neq 0, \quad \forall i = \overline{1, n}$, atunci se împarte fiecare ecuaţie de ordin i cu a_{ii}, iar astfel obţinem sistemul echivalent

$$x_i = \frac{b_i}{a_{ii}} - \frac{a_{i1}}{a_{ii}} \cdot x_1 - ... - \frac{a_{i,i-1}}{a_{ii}} \cdot x_{i-1} - \frac{a_{i,i+1}}{a_{ii}} \cdot x_{i+1} - ... - \frac{a_{in}}{a_{ii}} \cdot x_n, \quad i = \overline{1, n}.$$

Notând

$$\alpha = \begin{pmatrix} 0 & -\frac{a_{12}}{a_{11}} & ... & -\frac{a_{1n}}{a_{11}} \\ -\frac{a_{21}}{a_{22}} & 0 & ... & -\frac{a_{2n}}{a_{22}} \\ ... & ... & ... & ... \\ -\frac{a_{n1}}{a_{nn}} & -\frac{a_{n2}}{a_{nn}} & ... & 0 \end{pmatrix} \quad \text{şi} \quad \beta = \begin{pmatrix} \frac{b_1}{a_{11}} \\ ..... \\ \frac{b_n}{a_{nn}} \end{pmatrix}$$

obţinem noua formă a sistemului,

$$x = \alpha \cdot x + \beta$$

care conduce la construirea şirului iterativ

$$x^{(k+1)} = \alpha \cdot x^{(k)} + \beta, \quad k = 0, 1, 2, ...$$

Scierea pe componente a acestui proces iterativ este:

$$x_i^{(k+1)} = \sum_{j=1}^{n} \alpha_{ij} \cdot x_j^{(k)} + \beta_i = \sum_{j=1, j\neq i}^{n} \left(-\frac{a_{i,j}}{a_{i.i}} \right) \cdot x_j^{(k)} + \frac{b_i}{a_{ii}} \quad , \quad i = \overline{1, n}. \tag{2.1}$$

Dacă $a_{ii} \neq 0, \quad \forall i = \overline{1, n}$, atunci sistemul liniar are soluţie unică, iar şirul iterativ de mai sus este convergent către această soluţie.

Algoritmul metodei lui Jacobi:

I. Date de intrare: n numărul de ecuaţii şi de necunoscute
matricea sistemului $A = (a_{ij})_{i,j=\overline{1,n}}$
vectorul termenilor liberi $b = (b_1, ..., b_n)$
$\varepsilon > 0$ eroarea admisă

II. Date de ieşire: k, ultima iteraţie, $x[i, k]$, $i = \overline{1, n}$, aproximaţia soluţiei la ultima iteraţie
sau un mesaj de eroare

III. Paşii algoritmului

1. Se citesc n şi ε
2. Se citeşte matricea A şi vectorul $b : a[i, j]$, $i, j = \overline{1, n}$, $b[i]$, $i = \overline{1, n}$
3. Se verifică condiţia de convergenţă:{
   Pentru $i = \overline{1, n}$
   $U[i] := 0$
   Pentru $j = \overline{1, n}, j \neq i$ calculează $U[i] := U[i] + |a[i, j]|$
   Dacă $|a[i, i]| \leq U[i]$ atunci tipăreşte mesajul " Nu se poate aplica metoda lui Iacobi "; Stop
   }
4. Pentru $i = \overline{1, n}$ calculează $x[i, 0] := \frac{b[i]}{a[i,i]}$
5. Pentru $k \geq 1$ calculează
   Pentru $i = \overline{1, n}$ calculează
   $S[i] := 0$
   Pentru $j = \overline{1, n}$ calculează
   dacă $j \neq i$ execută $S[i] := S[i] + a[i, j] \cdot x[j, k-1]$
   $x[i, k] := (b[i] - S[i]) / a[i, i]$
   Cât timp
   $$\max_{i=\overline{1,n}} |x[i, k] - x[i, k-1]| \geq \varepsilon$$
6. Tipăreşte k
   Pentru $i = \overline{1, n}$ tipăreşte $x[i, k]$; Stop.

Exemplu numeric:

Cu $\varepsilon = 10^{-3}$ să se aplice algoritmul lui Iacobi (de mai sus) pentru rezolvarea sistemului liniar,

$$\begin{cases} 5x_1 + x_2 + x_3 = 10 \\ x_1 + 6x_2 + 4x_3 = 4 \\ x_1 + x_2 + 10x_3 = -7. \end{cases}$$

Pentru verificare, soluţia exactă este

$$x_1 = 2, \quad x_2 = 1, \quad x_3 = -1.$$

Metoda lui Gauss-Seidel

Dacă la fiecare iteraţie k utilizăm termenii vectorului $x^{(k)}$ calculaţi anterior în cadrul acestei iteraţii, se obţine o modificare a metodei lui Iacobi, rezultând algoritmul metodei Gauss-Seidel. Condiţia de convergenţă este aceeaşi: $|a_{i,i}| > \sum_{j=1, j\neq i}^{n} |a_{i,j}|$, $\forall i = \overline{1,n}$.

Scrierea iterativă a şirului $\left(x^{(k)}\right)_{k\in\mathbb{N}}$, $x^{(k+1)} = \alpha \cdot x^{(k)} + \beta, \quad k \in \mathbb{N}$, devine în acest caz:

1. pentru $i = \overline{1,n}$ calculează $x_i^{(0)} := \frac{b_i}{a_{ii}}$,
2. pentru $k \in \mathbb{N}$, calculează

$$x_1^{(k+1)} := \frac{b_1 - \sum_{j=2}^{n} a_{1j} \cdot x_j^{(k)}}{a_{11}} \tag{2.2}$$

$$x_2^{(k+1)} := \frac{b_2 - a_{21} \cdot x_1^{(k+1)} - \sum_{j=3}^{n} a_{2j} \cdot x_j^{(k)}}{a_{22}} \tag{2.3}$$

.....

$$x_i^{(k+1)} := \frac{b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(k)}}{a_{ii}} \tag{2.4}$$

.....

$$x_n^{(k+1)} := \frac{b_n - \sum_{j=1}^{n-1} a_{nj} \cdot x_j^{(k+1)}}{a_{nn}} \tag{2.5}$$

Observaţie: Formulele (2.2), (2.3), (2.4), (2.5) se scriu în mod concentrat astfel:

calculează

$$x_1^{(k+1)} := \frac{b_1 - \sum_{j=2}^{n} a_{1j} \cdot x_j^{(k)}}{a_{11}}$$

iar pentru $i = \overline{2,n}$ calculează

$$x_i^{(k+1)} := \frac{b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(k)}}{a_{ii}}. \tag{2.6}$$

Pentru implementarea algoritmului, la această metodă iterativă,

$$x_1^{(k+1)} := \frac{b_1 - \sum_{j=2}^{n} a_{1j} \cdot x_j^{(k)}}{a_{11}}, \quad k = 0, 1, 2, ...$$

iar formula (2.1) devine

$$x_i^{(k+1)} := \frac{b_i - \sum_{j=1}^{i-1} a_{ij} \cdot x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij} \cdot x_j^{(k)}}{a_{ii}}, \quad i = \overline{2,n}$$

cu $x_i^{(0)} := \frac{b_i}{a_{ii}}$, $i = \overline{1,n}$.

Algoritmul metodei Gauss-Seidel

I. Date de intrare: n numărul de ecuaţii şi de necunoscute
matricea sistemului $A = (a_{ij})_{i,j=\overline{1,n}}$
vectorul termenilor liberi $b = (b_1, ..., b_n)$
$\varepsilon > 0$ eroarea admisă

II. Date de ieşire: k, ultima iteraţie, $x[i,k]$, $i = \overline{1,n}$, aproximaţia soluţiei la ultima iteraţie
sau un mesaj de eroare

III. Paşii algoritmului

1. Se citesc n şi ε
2. Se citeşte matricea A şi vectorul b : $a[i,j]$, $i,j = \overline{1,n}$, $b[i]$, $i = \overline{1,n}$
3. Se verifică condiţia de convergenţă:

{

Pentru $i = \overline{1,n}$
$U[i] := 0$
Pentru $j = \overline{1,n}, j \neq i$ calculeaza $U[i] := U[i] + |a[i,j]|$
Daca $|a[i,i]| \leq U[i]$ atunci tipăreşte mesajul " Nu se poate aplica metoda lui Iacobi "; Stop

}

4. Pentru $i = \overline{1,n}$ calculează $x[i,0] := \frac{b[i]}{a[i,i]}$
5. Pentru $k \geq 1$ calculează

$V := 0;$
Pentru $j = \overline{1,n}$ calculează
$V := V + a[1,j] \cdot x[j,k-1]$
$x[1,k] := (b[1] - V)/a[1,1]$
Pentru $i = \overline{2,n}$ calculează
$S[i] := 0;$
Pentru $j = \overline{1,i-1}$ calculează
$S[i] := S[i] + a[i,j] \cdot x[j,k]$
Pentru $j = \overline{i+1,n}$ calculează
$S[i] := S[i] + a[i,j] \cdot x[j,k-1]$
$x[i,k] := (b[i] - S[i])/a[i,i]$
Cât timp

$$\max_{i=\overline{1,n}} |x[i,k] - x[i,k-1]| \geq \varepsilon$$

6. Tipăreşte k
Pentru $i = \overline{1,n}$ tipăreşte $x[i,k]$; Stop.

Exemplu numeric:
Cu $\varepsilon = 10^{-3}$ să se aplice algoritmul Gauss-Seidel pentru rezolvarea sistemului liniar,

$$\begin{cases} 5x_1 + x_2 + x_3 = 10 \\ x_1 + 6x_2 + 4x_3 = 4 \\ x_1 + x_2 + 10x_3 = -7. \end{cases}$$

Pentru verificare, soluţia exactă este

$$x_1 = 2, \quad x_2 = 1, \quad x_3 = -1.$$

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru cele două procedee prezentate în acest laborator, rezolvările sistemelor sunt foarte diferite decât procedeele anterioare, dar ca și bază, acestea au aceleași proprietăți ale sistemului pe care îl rezolvăm. Așa că putem refolosi aceeași clasă de bază și pentru aceste procedee, și mai adăugăm proprietățile noi pentru rezolvarea iterativă: eroarea *epsilon* admisă la aproximare, pasul k incrementat în timpul rezolvării algoritmului, și un "istoric" al valorilor șirului x, notat cu X (alternativ, în funcție de cum doriți să rezolvați sistemul, se pot memora doar valorile șirului x de la pasul anterior într-o variabilă *x2*). Doar *epsilon* va fi primit ca parametru în consturctor, și inițializăm X (sau *x2*).

Deoarece pașii celor două procedee sunt aceiași, și doar calcularea sumelor diferă, propunem crearea unei metode virtuale *CalculeazaSume*, inițial în clasa metodei lui Jacobi având calculul pentru acest procedeu, iar în clasa derivata din Jacobi, pentru procedeul Gauss-Seidel, să suprascriem calculul sumelor cu noua valoare, din acest procedeu. Astfel, în clasa *MetodaGaussSeidel*, nu mai este nevoie deloc să suprascriem metoda *Rezolvare*.

Algorithm 6 Clasa MetodaJacobi

```
1: public class MetodaJacobi :  Sistem
2: {
3:     public decimal epsilon;
4:     public int k;
5:     public List<decimal[]> X;
6:     public MetodaJacobi(int n, decimal[,] A, decimal[] b,
7:         decimal epsilon) :  base(n, A, b)
8:     {
9:         this.epsilon = epsilon;
10:        X = new List<decimal[]>();
11:    }
12:    public override void Rezolvare()
13:    {
14:        // Algoritm
15:    }
16:    protected virtual void CalculeazaSume(decimal[] s, int i)
17:    {
18:        for(int j = 0; j < n; j++)
19:            if(j != i)
20:                s[i] = s[i] + A[i, j] * X[k - 1][j];
21:    }
22: }
```

Algorithm 7 Clasa MetodaGaussSeidel

```
1: public class MetodaGaussSeidel :  MetodaJacobi
2: {
3:     public MetodaGaussSeidel(int n, decimal[,] A, decimal[] b,
4:         decimal epsilon) :  base(n, A, b, epsilon) { }
5:     protected override void CalculeazaSume(decimal[] s, int i)
6:     {
7:         // Formula diferita pentru calcularea sumelor
8:     }
9: }
```

Capitolul 3

Laboratorul 3. Ecuaţii neliniare. Metoda coardei

METODE ITERATIVE PENTRU REZOLVAREA ECUAȚIILOR NELINIARE

Metoda înjumătățirii intervalului și metoda coardei

Metoda înjumătățirii intervalului

Algoritmul metodei înjumătăţirii intervalului
Algoritmul aproximează soluţia izolată $x^* \in [a,b]$ a ecuaţiei $f(x)=0$, $f \in C[a,b]$, prin metoda înjumătăţirii intervalului.

I. Date de intrare: expresia funcţiei f
capetele intervalului: a, b
precizia dorită: *eps*

II. Date de ieşire: Aproximarea rădăcinii ecuaţiei: SOL

III. Paşii algoritmului

1. Fie $c := \frac{a+b}{2}$
2. Dacă $b-c < eps$ atunci SOL:= c; Stop.
altfel
Dacă $f(b) \cdot f(c) < 0$ atunci $a := c$
altfel $b := c$
3. Mergi la pasul 1.
4. Tipăreste SOL; Stop.

Exemplu numeric: Se aproximează soluția reală a ecuației $x^3 - x - 1 = 0$, luând $a = 1$, $b = 2$, $f(x) = x^3 - x - 1$ și $eps = 10^{-4}$.

Metoda coardei

Se construieşte şirul iterativ $(x_n)_{n\geq 0}$, convergent către unica soluţie $x^* \in (a,b)$, a ecuaţiei $f(x)=0$ folosind una din formulele

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f(b)-f(x_{n-1})}(b-x_{n-1}), \qquad x_0 = a,$$

dacă $f(a)\cdot f''(a) < 0$, sau

$$x_n = x_{n-1} - \frac{f(x_{n-1})}{f(a)-f(x_{n-1})}(a-x_{n-1}), \qquad x_0 = b$$

dacă $f(b)\cdot f''(b) < 0$.

Algoritm pentru metodei coardei:

I. Date de intrare: a, b capetele intervalului
$\varepsilon > 0$ eroarea admisă
expresia funcţiei f
valorile $f''(a) = dda$, $f''(b) = ddb$ necesare pentru alegerea primului termen de iteraţie

II. Date de ieşire: n ultima iteraţie
x_n aproximaţia soluţiei la ultima iteraţie, sau şirul de aproximaţii $x_i, i = \overline{0,n}$

III. Paşii algoritmului

1. Se introduc datele a, b, ε
2. Se defineşte expresia funcţiei f
3. Se introduc valorile dda, ddb
4. Dacă $f(a)\cdot dda < 0$ atunci $x(0) := a$
   4.1. Fie
   $$x_1 := x_0 - \frac{f(x_0)}{f(b)-f(x_0)}\cdot(b-x_0)$$
   4.2. Pornind cu $n \geq 1$, cât timp
   $$|x_n - x_{n-1}| \geq \varepsilon$$
   Calculează
   $$x_{n+1} := x_n - \frac{f(x_n)}{f(b)-f(x_n)}\cdot(b-x_n)$$
   4.3. Tipăreşte n, x_n. STOP.
5. Dacă $f(b)\cdot ddb < 0$ atunci $x(0) := b$
   5.1. Fie
   $$x_1 = x_0 - \frac{f(x_0)}{f(a)-f(x_0)}\cdot(a-x_0)$$
   5.2. Pornind cu $n \geq 1$, cât timp
   $$|x_n - x_{n-1}| \geq \varepsilon$$

Calculează

$$x_{n+1} = x_n - \frac{f(x_n)}{f(a) - f(x_n)} \cdot (a - x_n)$$

5.3. Tipăreşte n, x_n. STOP.

Exemple numerice:

1. Aproximarea soluției reale x^* a ecuației $x^3 - 3x + 1 = 0$ situate în intervalul $(0, 1)$. Se ia $a = 0$, $b = 1$, $f(x) = x^3 - 3x + 1$ și $\varepsilon = 10^{-4}$, iar $dda = 0$, $ddb = 6$.

2. Aproximarea soluției reale $x^* \in (1, 2)$ a ecuației $x^3 - x - 1 = 0$.Se ia $a = 1$, $b = 2$, $f(x) = x^3 - x - 1$ și $\varepsilon = 10^{-4}$, $dda = 6$, $ddb = 12$.

3. Aproximarea numărului $\sqrt{2}$: se va considera $a = 1$, $b = 2$, $f(x) = x^2 - 2$ și $\varepsilon = 10^{-4}$, $dda = 2$, $ddb = 2$.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Deoarece trecem de la metode pentru rezolvarea unui sistem de ecuaţii liniare, la alte metode care implică aflarea soluţiei prin mai multe iteraţii ale unui şir, avem nevoie de o nouă metodă abstractă ca şi clasă de bază a procedeelor pe care urmează să le implementăm.

Atât metoda înjumătăţirii intervalului, cât şi metoda coardei au elemente comune: funcţia $f(x)$, capetele intervalului a şi b, precizia dorită ε, x o listă dinamică de valori care reţine valorile şirului, respectiv numărul de iteraţii. Pentru metoda coardei mai sunt necesare valorile celei de a doua derivate la capete, $f''(a)$ şi $f''(b)$, pentru alegerea primului termen de iterare.

Constructorul clasei primeşte datele de intrare, se iniţializează lista de valori x şi numărul de *Iteratii* cu 0. Metoda abstractă *Rezolvare()* va fi implementată separat în fiecare clasă derivată, corespunzător algoritmului specific fiecărei metode. Astfel, codul rămâne clar, organizat şi uşor de întreţinut.

Algorithm 8 Clasa de bază MetodaNumerica

```
public abstract class MetodaNumerica
{
    public Func <decimal, decimal> f;
    public decimal a, b;
    public decimal eps;
    public List<decimal> x;
    public int Iteratii;
    public MetodaNumerica(Func <decimal, decimal> f, decimal a,
        decimal b, decimal eps)
    {
        this.f = f;
        this.a = a;
        this.b = b;
        this.eps = eps;
        x = new List<decimal>();
        this.Iteratii = 0;
    }
    public abstract void Rezolvare();
}
```

Pentru fiecare din cele două metode - metoda înjumătăţirii intervalului şi metoda coardei, se va crea o clasă derivată din clasa MetodaNumerica.

Constructorul clasei MetodaCoardei, primeşte în plus ca parametrii variabilele *dda* şi *ddb*, deoarece metoda coardei are nevoie de derivata a doua la capetele intervalului pentru a decide punctul iniţial şi apoi iterează după regula coardei.

Algorithm 9 Clasa MetodaInjumatatiriiIntervalului

```
public class MetodaInjumatatiriiIntervalului :  MetodaNumerica
{
    public MetodaInjumatatiriiIntervalului(Func <decimal, decimal> f,
        decimal a, decimal b, decimal eps) :  base(f, a, b, eps) { }
    public override void Rezolvare()
    {
        // Algoritm
    }
}
```

Algorithm 10 Clasa MetodaCoardei

```
public class MetodaCoardei :  MetodaNumerica
{
    public decimal dda, ddb;
    public MetodaCoardei(Func <decimal, decimal> f, decimal a,
        decimal b, decimal eps, decimal dda, decimal ddb)
        :  base(f, a, b, eps)
    {
        this.dda = dda;
        this.ddb = ddb;
    }
    public override void Rezolvare()
    {
        // Algoritm
    }
}
```

Pentru verificarea exemplelor, în metoda *Main()*, iniţializăm clasa folosind valorile definite în exemplu.

Algorithm 11 Folosirea claselor în metoda Main()

```
static void Main(string[] args)
{
    decimal a = 0;
    decimal b = 1;
    decimal eps = 0.0001m;
    Func< decimal, decimal> f = F;

    MetodaInjumatatiriiIntervalului injumatatiriiIntervalului = new
    MetodaInjumatatiriiIntervalului(f, a, b, eps);
    injumatatiriiIntervalului.Rezolvare();

    Console.WriteLine("Metoda injumatatirii intervalului:");
    Console.WriteLine($"Rezultat:
    {injumatatiriiIntervalului.x[injumatatiriiIntervalului.Iteratii]}
    dupa {injumatatiriiIntervalului.Iteratii} iteratii.  \n");
}
static decimal F(decimal x)
{
    return x * x * x - 3 * x + 1;
}
```

Am creat metoda $F(x)$, în afara metodei $Main()$. Este nevoie ca metoda $F(x)$ să fie statică, deoarece şi metoda $Main()$ este statică şi nu ar putea folosi metoda $F(x)$ direct altfel.

Capitolul 4

Laboratorul 4. Metoda tangentei

METODA TANGENTEI. CALCULUL RADICALILOR. METODA MODIFICATĂ A TANGENTEI

Metoda tangentei

Pentru aproximarea soluției ecuației $f(x) = 0$, $x \in [a, b]$, prin metoda tangentei se construiește șirul iterativ

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad n \in \mathbb{N}$$

alegându-se iterația inițială

$$x_0 = \begin{cases} a, & \text{dacă } f(a) \cdot f''(a) > 0 \\ b, & \text{dacă } f(b) \cdot f''(b) > 0. \end{cases}$$

Algoritmul metodei tangentei (a lui Newton)

Algoritmul calculează o aproximaţie a soluţiei izolate $x^* \in (a, b)$ a ecuaţiei $f(x) = 0$, $f \in C[a, b]$, prin metoda lui Newton.

I. Date de intrare:

a, b-capetele intervalului (date de tip *double*)

expresia funcţiei f (de argument real, declarată ca dată de tip *double*)

df- expresia derivatei funcţiei f

eps- precizia dorită (dată de tip *double*)

dda - valoarea derivatei a doua f'' în a (dată de tip *double*)

II. Date de iesire: n, $x[n]$

III. Pașii algoritmului

1: Dacă $f(a) \cdot dda > 0$ atunci $x[0] := a$

altfel $x[0] := b$

2: Calculează

$$x[1] = x[0] - \frac{f(x[0])}{df(x[0])}$$

3: Pornind cu $n \geq 1$, cât timp : $|x[n] - x[n-1]| \geq eps$
Calculează

$$x[n+1] = x[n] - \frac{f(x[n])}{df(x[n])}$$

4. Tipărește n. Tipăreste $x[n]$. Stop.

Exemple numerice:

1. Folosind metoda tangentei să se aproximeze rădăcina reală a ecuației

$$x^3 - x - 1 = 0$$

considerând $f(x) = x^3 - x - 1$, $df(x) = 3x^2 - 1$, $a = 1$, $b = 2$, $dda = 6$ și luând succesiv $\varepsilon = 10^{-4}$, $\varepsilon = 10^{-8}$, $\varepsilon = 10^{-12}$, pentru a vedea numărul de iterații necesare.

2. Folosind metoda tangentei să se aproximeze rădăcina reală a ecuației

$$x^5 - 5x + 1 = 0$$

situată în intervalul $(-1, 1)$. Se va lua $f(x) = x^5 - 5x + 1$, $df(x) = 5x^4 - 5$, $a = 0$, $b = 1$, $dda = 11$ și $\varepsilon = 10^{-4}$. Ce constatați ?

Aproximarea radicalilor

Pentru a aproxima $\sqrt[k]{c}$ se poate adapta metoda tangentei folosind funcția $f : [0, b] \to \mathbb{R}$ definită prin $f(x) = x^k - c$, unde $b = \max\{1, c\}$. Se obține șirul recurent

$$x_n = x_{n-1} - \frac{x_{n-1}^k - c}{k x_{n-1}^{k-1}} = \frac{1}{k}\left[(k-1)\, x_{n-1} + \frac{c}{x_{n-1}^{k-1}}\right], \quad n \in \mathbb{N}^*$$

luând $x_0 = b$.

La implementarea algoritmului se va lua $f(x) = x^k - c$ ți $df(x) = kx^{k-1}$, iar pasul 1 se va modifica astfel:

"*Dacă* $c > 1$ *atunci* $x[0] := c$
altfel $x[0] := 1$"

Exemple numerice:

Să se aproximeze valorile radicalilor $\sqrt{2}$, $\sqrt{3}$, $\sqrt{5}$, cu metoda tangentei. Să se aproximeze $\sqrt[3]{2}$, $\sqrt[3]{3}$, și $\sqrt[3]{\frac{1}{2}}$ prin metoda tangentei.

Metoda modificată a tangentei

În loc de a calcula la fiecare iterație valoarea $f'(x_n)$, se poate utiliza șirul iterativ

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_0)}, \quad n \in \mathbb{N}$$

generat de metoda modificată a tangentei în care la fiecare iterație se folosește valoarea $f'(x_0)$ calculată pentru prima iterație, nefiind necesară definirea expresiei primei derivate.

Algoritmul metodei modificate a tangentei

I. Date de intrare:

a, b-capetele intervalului (date de tip *double*)

expresia funcţiei f (de argument real, declarată ca dată de tip *double*)

eps- precizia dorită (dată de tip *double*)

dda - valoarea derivatei a doua f'' în a (dată de tip *double*)

II. Date de iesire: n, $x[n]$

III. Pașii algoritmului

1: Dacă $f(a) \cdot dda > 0$ atunci $x[0] := a$

altfel $x[0] := b$

2: Se precizează valoarea primei derivate pe iterația inițială $d0 := f'(x_0)$

3: Calculează

$$x[1] = x[0] - \frac{f(x[0])}{d0}$$

4: Pornind cu $n \geq 1$, cât timp : $|x[n] - x[n-1]| \geq eps$

Calculează

$$x[n+1] = x[n] - \frac{f(x[n])}{d0}$$

5. Tipărește n. Tipărește $x[n]$. Stop.

Exemplu numeric:

Se consideră $f(x) = x^3 - x - 1$, $a = 1$, $b = 2$, $dda = 6$ și $d0 = 11$. Se va lua succesiv $\varepsilon = 10^{-4}$, $\varepsilon = 10^{-8}$, $\varepsilon = 10^{-12}$, pentru a vedea numărul de iterații necesare la fiecare alegere a erorii de toleranță ε.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru aproximarea soluţiei unei ecuaţii neliniare $f(x) = 0$, folosind metoda tangentei, metoda calculului radicalilor, respectiv metoda tangentei modificată folosim aceeaşi metodă abstractă ca şi clasă de bază MetodaNumerica. Cele trei metode au aceleaşi proprietăţi ale clasei: funcţia $f(x)$, intervalul $[a, b]$ în care se caută soluţia, precizia dorită ε şi x o listă dinamică de valori care reţine valorile şirului, respectiv numărul de iteraţii.

Implementarea metodei tangentei este realizată prin clasa derivată MetodaTangentei. Constructorul clasei primeşte în plus funcţia derivată $f'(x)$ şi valoarea derivatelor de ordinul doi la capete, folosită pentru alegerea punctului iniţial. Derivata funcţiei în capătul stâng va fi salvată în proprietatea *df*, iar valoarea derivatei a doua în capătul stâng va fi salvată în variabila $ddfa$.

Algorithm 12 Clasa MetodaTangentei

```
public class MetodaTangentei :  MetodaNumerica
{
    private Func <decimal, decimal> df;
    private decimal ddfa;
    public MetodaNumerica(Func <decimal, decimal> f, decimal a,
        decimal b, decimal eps, Func<decimal, decimal> df,
        decimal ddfa) :  base (f, a, b, eps)
    {
        this.df = df;
        this.ddfa = ddfa;
    }
    public override void Rezolvare()
    {
        // Algoritm
    }
}
```

Calculul radicalilor de ordin m dintr-un număr c este o particularizare a metodei tangentei. Aceasta este implementată prin clasa derivată MT_Radical, care extinde MetodaTangentei.

Aici, $f(x)$ şi $f'(x)$ sunt particularizate: $f(x) = x^m - c$, $f'(x) = m \cdot x^{m-1}$.
Valoarea lui c ajută şi la alegerea intervalului şi a punctului iniţial. Astfel, va trebui doar să se indice ordinul radicalului şi numărul pentru care se face calculul.

La metoda tangentei modificată, diferenţa care apare este în constructorul clasei MetodaTangenteiModificata care în metoda Rezolvare() unde iterarea şirului se face după formula

$$x_{n+1} = x_n - f(x_n)/f'(x_0).$$

Singura diferenţa între metoda tangentei şi metoda tangentei modificată este că formula pasului iterativ nu mai calculează valoarea derivatei funcţiei folosind soluţia pasului ante-

Algorithm 13 Clasa MT_Radical

```
public class MT_Radical :  MetodaTangentei
{
    private int m;
    private decimal c;
    public MT_Radical(decimal eps, int m, decimal c)
        :  base (a:  (c>1 ?  1 :  0),   b:  c,   eps:  eps,
            f:  x => x.LaPuterea(m) - c,
            df:  x => m * x.LaPuterea(m - 1),
            ddfa:  (c>1 ?  m * (m-1) :  0) )
    {
        this.m = m;
        this.c = c;
    }
}
```

rior, ci se folosește doar de derivata funcţiei în punctul x_0. Pentru a nu rescrie metoda Rezolvare() putem da valoare lui *df* o funcţie care returnează întotdeauna valoarea derivatei funcţiei în punctul x_0. Această valoare va fi parametrul $dfx0$ transmis constructorului noii clase MetodaModificataATangentei, iar când vom apela constructorul clasei de bază MetodaTangentei, pentru df vom trimite

$$x => dfx0.$$

Algorithm 14 Clasa MetodaModificataATangentei

```
public class MetodaModificataATangentei :  MetodaTangentei
{
    public MetodaModificataATangentei(decimal a, decimal b,
        decimal epsilon, Func<decimal, decimal> f, decimal dfx0,
        decimal ddfa) :  base(a, b, epsilon, f, x => dfx0, ddfa)
    { }
}
```

În metoda Main(), iniţializăm clasele folosind valorile definite în exemplu.

Algorithm 15 Folosirea claselor în metoda Main()

```
static void Main(string[] args)
{
    decimal a = 1;
    decimal b = 2;
    decimal eps = 1e-28M;
    decimal ddfa = 6;
    Func<decimal, decimal> f = x => x * x * x - x - 1;
    Func<decimal, decimal> df = x => 3 * x * x - 1;

    var tangenta = new MetodaTangentei(f, a, b, eps, df, ddfa);
    tangenta.Rezolvare();

    Console.WriteLine("Metoda tangentei:");
    Console.WriteLine($"Rezultat:  {tangenta.x[tangenta.Iteratii]}
            după {tangenta.Iteratii} iterații.");

    Console.WriteLine("Metoda Tangentei - Calcularea Radicalilor");
    var rad = new MT_Radical(1e-28M, 2, 2);
    rad.Rezolvare();
    Console.WriteLine(rad);
}
```

Capitolul 5

Laboratorul 5. Metoda aproximaţiilor succesive

METODA APROXIMAȚIILOR SUCCESIVE PENTRU ECUAȚII NELINIARE

Metoda aproximațiilor succesive

Pentru rezolvarea numerică a ecuaţiei $f(x) = 0$, cu $f : I \to \mathbb{R}$, iar $I \subset \mathbb{R}$ interval, prin metoda aproximaților succesive, se rescrie ecuația sub forma echivalentă $\varphi(x) = x$ pe un subinterval $[a, b] \subset I$ pe care este îndeplinită condiția $|\varphi'(x)| \leq q < 1$, $\forall x \in [a, b]$. Astfel, se poate construi prin recurență șirul aproximațiilor succesive

$$x_{n+1} = \varphi(x_n), \quad n \in \mathbb{N} \tag{5.1}$$

care converge către unica soluție $x^* \in (a, b)$ a ecuației $\varphi(x) = x$. Din inegalitatea de estimare a posteriori a erorii,

$$|x_n - x^*| \leq \frac{q}{1-q} \cdot |x_n - x_{n-1}|, \quad \forall n \in \mathbb{N}^*$$

se deduce următorul criteriu de oprire a algoritmului iterativ dat în formula (5.1):

Pentru $\varepsilon > 0$ dat se determină primul număr natural $n \in N^*$ pentru care are loc inegalitatea

$$|x_n - x_{n-1}| < \varepsilon$$

și se reține termenul x_n ca aproximație a soluției x^*.

La acest termen estimarea erorii este:

$$|x_n - x^*| < \frac{q}{1-q} \cdot \varepsilon.$$

Algoritmul metodei aproximațiilor succesive

I. Date de intrare:
x_0 iterația inițială (dată de tip double)
φ funcția de variabilă de tip double
$\varepsilon > 0$ eroarea admisă (dată de tip double)
II. Date de ieșire: n numărul de iterații
x_n ultima iterație ce aproximează soluția cu eroarea admisă
III. Pașii algoritmului
1. Calculează

$$x_1 = \varphi(x_0)$$

2. Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon$$

Calculează

$$x_{n+1} = \varphi(x_n)$$

3. Tipărește n, x_n. STOP.
Exemple numerice:
1. Folosind metoda aproximațiilor succesive să se aproximeze soluția ecuației

$$x^4 - x - 1 = 0$$

situată în intervalul $(1, 2)$. Se va lua $\varepsilon = 10^{-4}$. Ecuația se va pune în forma echivalentă

$$x = \varphi(x) = \sqrt[4]{x+1}.$$

2. Folosind metoda aproximațiilor succesive să se aproximeze rădăcina reala a ecuației

$$x^3 - x - 1 = 0$$

luând succesiv $\varepsilon = 10^{-4}$, $\varepsilon = 10^{-8}$, $\varepsilon = 10^{-12}$, pentru a vedea numărul de iterații necesare în fiecare caz. Se va lua $\varphi(x) = \sqrt[3]{x+1}$ si $x_0 = 1$.

Explicație: Considerând $f(x) = x^3 - x - 1$, avem $f'(x) = 3x^2 - 1$, iar rădăcinile derivatei sunt $x = \pm\frac{1}{\sqrt{3}}$. Deoarece $f\left(-\frac{1}{\sqrt{3}}\right) < 0$, $f\left(\frac{1}{\sqrt{3}}\right) < 0$, $\lim_{x\to-\infty} f(x) = -\infty$, și $\lim_{x\to\infty} f(x) = \infty$, deducem că ecuația are o singură rădăcină reală (celelalte două rădăcini sunt complexe conjugate) situată în intervalul $\left(\frac{1}{\sqrt{3}}, \infty\right)$. Acest interval fiind prea mare vom testa semnul valorilor $f(1)$ si $f(2)$, obținând $f(1) = -1$ si $f(2) = 5$. Deci, rădăcina reală se gaseste în intervalul $(1, 2)$. Încercând să rescriem ecuația sub forma $x = \varphi(x)$ observăm că reprezentarea echivalentă $x = x^3 + 1 = \varphi(x)$ nu este potrivită deoarece $\varphi'(x) = 3x^2$ și pe intervalul $[1, 2]$ avem $|\varphi'(x)| \geq 3, \forall x \in [1, 2]$. Prin urmare, din rescrierea ecuației sub forma $x^3 = x + 1$, folosind bijectivitatea pe intervalul $[1, 2]$ a ambelor funcții $g_1(x) = x^3$ si $g_2(x) = x + 1$, ajungem la egalitatea $x = \sqrt[3]{x+1}$. Deci, se va considera forma echivalentă a ecuației:

$$x = \varphi(x) = \sqrt[3]{x+1}$$

având $\varphi'(x) = \frac{1}{3\sqrt[3]{(x+1)^2}} \leq \frac{1}{3\sqrt[3]{4}} < 1$ și putem lua ca iterație inițială orice punct din intervalul $[1,2]$, de exemplu $x_0 = 1$.

3. Folosind metoda aproximațiilor succesive să se aproximeze rădăcina reală a ecuației

$$f(x) = x^5 - 5x + 1 = 0$$

situată în intervalul $(0,1)$. Se va lua $\varphi(x) = \frac{1}{5} \cdot (x^5 + 1)$, $x_0 = \frac{1}{2}$ și $\varepsilon = 10^{-4}$.

Explicație: Din sirul de egalități

$$x^5 - 5x + 1 = 0 \Longleftrightarrow 5x = x^5 + 1 \Longleftrightarrow x = \frac{1}{5} \cdot \left(x^5 + 1\right)$$

deducem că ecuația se va pune în forma echivalentă

$$x = \varphi(x) = \frac{1}{5} \cdot \left(x^5 + 1\right)$$

și atunci $\varphi'(x) = x^4$ putând avea îndeplinită condiția $|\varphi'(x)| \leq q < 1$ pe intervalul $(0,1)$. Deoarece $f(0) = 1$, $f\left(\frac{1}{2}\right) = -\frac{47}{32}$ și $f(1) = -3$ deducem că rădăcina ecuației se găsește în intervalul $\left(0, \frac{1}{2}\right)$. Se poate considera iterația inițială $x_0 = \frac{1}{2}$.

4. Folosind metoda aproximațiilor succesive să se aproximeze rădăcina reală a ecuației lui Kepler (ce apare în astronomie):

$$x = \sin x + 0.25$$

situată în intervalul $\left(\frac{\pi}{4}, \frac{\pi}{2}\right)$. Rădăcina se izolează folosind metoda șirului lui Rolle și înjumătățirea intervalului, ajungând la iterația inițială $x_0 = \frac{3\pi}{8} \simeq 1.1781$. Se va lua $\varphi(x) = \sin x + 0.25$, $\varepsilon = 10^{-4}$ și aplicând procedeul recurent (5.1) se va obține $x_5 = 1.172$ cu o eroare mai mică decât 0.0016.

Metoda lui Heron

Pentru $a > 0$ sirul dat prin,

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{a}{x_n}\right)$$

converge către $\sqrt{a}$. Într-adevăr, la aplicarea metodei aproximațiilor succesive putem utiliza funcția $\varphi(x) = \frac{1}{2}\left(x + \frac{a}{x}\right)$. Dacă $a > 1$ atunci se poate considera $\varphi : [\sqrt{a}, a] \longrightarrow \mathbb{R}$, ăi avem $0 \leq \varphi'(x) < \frac{1}{2} = q$, $\forall x \in [\sqrt{a}, a]$. Astfel, luând $x_0 = a$ obținem $\lim\limits_{n\to\infty} x_n = x^*$ și

$$x^* = \frac{1}{2}\left(x^* + \frac{a}{x^*}\right)$$

adică, $x^* = \sqrt{a}$. Dacă $a < 1$, atunci vom considera $\varphi : [\sqrt{a}, 1] \longrightarrow \mathbb{R}$, și în acest caz, $0 \leq \varphi'(x) \leq \frac{1-a}{2} \leq \frac{1}{2} = q < 1$, $\forall x \in [\sqrt{a}, 1]$. Luând $x_0 = 1$ obținem din nou $\lim\limits_{n\to\infty} x_n = x^* = \sqrt{a}$.

Observăm că atunci când $a > 10$ este indicat să alegem ca iterație inițială $x_0 = k$, unde k este număr întreg cu proprietatea

$$0 < k < \sqrt{a} < k + 1.$$

Algoritmul lui Heron

I. Date de intrare:

a numărul din care se extrage radicalul (dată de tip double)

x_0 iterația inițială (dată de tip double)

$\varepsilon > 0$ eroarea admisă (dată de tip double)

II. Date de ieșire: n numărul de iterații

x_n ultima iterație ce aproximează soluția cu eroarea admisă

III. Pașii algoritmului

1. Calculează $x[0] := x_0$ si

$$x[1] := 0.5 \cdot \left(x[0] + \frac{a}{x[0]} \right)$$

2. Pornind cu $n \geq 1$, cât timp

$$|x[n] - x[n-1]| \geq \varepsilon$$

Calculează

$$x[n+1] = 0.5 \cdot \left(x[n] + \frac{a}{x[n]} \right)$$

3. Tipărește n, $x[n]$; STOP.

Exemple numerice:

1. Să se aproximeze valorile radicalilor $\sqrt{2}$ $\sqrt{3}$, si $\sqrt{5}$ prin metoda lui Heron. Constanta a va lua pe rând valorile 2, 3 si 5. Se va lua ca iterație inițială $x_0 = a$ și se va considera succesiv $\varepsilon = 10^{-4}$, $\varepsilon = 10^{-8}$, $\varepsilon = 10^{-12}$, pentru a vedea numărul de iterații necesare în fiecare caz și evoluția termenilor din șirul aproximațiilor succesive.

2. Să se aproximeze $\sqrt{\frac{1}{2}}$ prin metoda lui Heron. Se va considera $a = \frac{1}{2}$ și iterația inițială $x_0 = 1$, luând succesiv $\varepsilon = 10^{-4}$, $\varepsilon = 10^{-8}$, $\varepsilon = 10^{-12}$, pentru a vedea numărul de iterații necesare în fiecare caz.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Clasa MetodaAproximatiilorSuccesive moștenește clasa de bază MetodaNumerica, reutilizând elementele comune tuturor metodelor iterative. Funcția de iterație $\varphi(x)$ se obține prin transformarea ecuației $f(x) = 0$ într-o formă echivalentă $x = \varphi(x)$, iar constructorul clasei MetodaAproximatiilorSuccesive inițializează intervalul de căutare $[a, b]$, precizia dorită ε, funcția $f(x)$, funcția de iterație $\varphi(x)$ și aproximația inițială x_0.

Metoda Exemple() conține câteva exemple de utilizare pentru diferite ecuații neliniare și afișează soluțiile obținute.

Algorithm 16 Clasa MetodaAproximatiilorSuccesive

```
public class MetodaAproximatiilorSuccesive :  MetodaNumerica
{
    protected override decimal x0;
    protected Func<decimal, decimal> fi;
    public MetodaAproximatiilorSuccesive(decimal a, decimal b,
        decimal epsilon, Func<decimal, decimal> f,
        Func<decimal, decimal> fi, decimal x0)
        :  base (a, b, epsilon, f)
    {
        this.fi = fi;
        this.x0 = x0;
    }
    public static void Exemple()
    {
        // Exemple
    }
    public static void Calculare()
    {
        // Algoritm
    }
}
```

Clasa MetodaHeron moștenește clasa MetodaNumerica și implementează algoritmul lui Heron pentru aproximarea rădăcinii pătrate. Constructorul clasei primește valoarea pentru care se calculează radicalul, precizia, functia f și aproximația inițială.

În metoda Exemple() se ilustrează calculul valorilor $\sqrt{2}$, $\sqrt{3}$ și $\sqrt{\frac{1}{2}}$, folosind algoritmul lui Heron.

Algorithm 17 Clasa Metoda lui Heron

```
public class MetodaHeron :  MetodaNumerica
{
    protected override decimal x0;
    public MetodaHeron(decimal a, decimal epsilon,
        Func<decimal, decimal> f, decimal x0)
        :  base (a, 0, epsilon, f)
    {
        this.x0 = x0;
    }
    public static void Exemple()
    {
        // Exemple
    }
    public static void Calculare()
    {
        // Algoritm
    }
}
```

Capitolul 6

Laboratorul 6. Sisteme neliniare. Metode iterative

Metoda aproximațiilor succesive pentru sisteme de ecuații neliniare

Metoda aproximațiilor succesive pentru sisteme neliniare

Algoritmul metodei aproximaţiilor succesive pentru sisteme neliniare
I. Date de intrare:
x_0 şi y_0 iteraţia iniţială (date de tip double)
F şi G funcţii de două variabile de tip double
$\varepsilon > 0$ eroarea admisă (dată de tip double)
II. Date de ieşire:
x_n şi y_n ultima iterație ce aproximează soluţia cu eroarea admisă
III. Paşii algoritmului
1. Calculează

$$\begin{cases} x_1 := F(x_0, y_0) \\ y_1 := G(x_0, y_0), \end{cases}$$

2. Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon \quad \text{sau} \quad |y_n - y_{n-1}| \geq \varepsilon$$

Calculează

$$\begin{cases} x_{n+1} := F(x_n, y_n) \\ y_{n+1} := G(x_n, y_n), \end{cases}$$

3. Tipăreşte n, x_n, y_n. STOP.

Exemple numerice:

1. Să se aproximeze soluția, cu ambele coordonate pozitive, a sistemului

$$\begin{cases} f(x,y) = 2x^2 - xy - 5x + 1 = 0 \\ g(x,y) = x + 3\lg x - y^2 = 0. \end{cases}$$

Se va considera

$$F(x,y) = \sqrt{\frac{x(y+5)-1}{2}}$$

și $G(x,y) = \sqrt{x + 3\lg x}$ și atunci sistemul devine

$$\begin{cases} x = \sqrt{\frac{x(y+5)-1}{2}} \\ y = \sqrt{x + 3\lg x} \end{cases}$$

luându-se ca prima aproximație punctul $(x_0, y_0) = (3.5, 2.2)$. Se va considera vecinătatea lui (x_0, y_0) ca fiind

$$V = \{(x,y) : |x - x_0| \leq 0.1, |y - y_0| \leq 0.1\}$$

și pe acest domeniu, vom avea $M = \max\{0.54 + 0.27, 0.42 + 0\} = 0.81 < 1$. Aplicând procedeul iterativ

$$\begin{cases} x_{n+1} := F(x_n, y_n) \\ y_{n+1} := G(x_n, y_n), \end{cases}$$

vom obține aproximația cu patru cifre semnificative exacte $(x_6, y_6) = (3.487, 2.262)$.

2. Să se verifice metoda pe sistemul simetric

$$\begin{cases} x^2 + y^2 = 5 \\ xy = 2 \end{cases}$$

care are soluţiile $(-2,-1), (-1,-2), (1,2), (2,1)$ şi care se scrie sub forma

$$\begin{cases} x = \sqrt{5 - y^2} = F(x,y) \\ y = \frac{2}{x} = G(x,y) \end{cases}$$

în vecinătatea V a punctului $(2,1)$,

$$V = \{(x,y) : |x - 2| \leq 0.1, |y - 1| \leq 0.1\},$$

unde sunt îndeplinite condiţiile din teorema 1. Se va lua $\varepsilon = 10^{-4}$.

Observație: Sistemul precedent are soluțiile $(2,1)$, $(1,2)$, $(-1,-2)$, $(-2,-1)$ și în vecinătatea fiecărei soluții se consideră câte o pereche de funcții F, G. Astfel, în vecinătatea punctului $(1,2)$ se ia $F(x,y) = \frac{2}{y}$ și $G(x,y) = \sqrt{5 - x^2}$. Pentru soluția $(-1,-2)$ avem $F(x,y) = \frac{2}{y}$ și $G(x,y) = -\sqrt{5 - x^2}$, iar pentru soluția $(-2,-1)$ avem $F(x,y) = -\sqrt{5 - y^2}$ și $G(x,y) = \frac{2}{x}$.

METODA GAUSS-SEIDEL PENTRU SISTEME DE ECUAȚII NELINIARE

Considerăm sistemul neliniar

$$\begin{cases} x = F_1(x, y, z) \\ y = F_2(x, y, z) \\ z = F_3(x, y, z) \end{cases}$$

pentru care șirul iterativ generat de metoda Gauss-Seidel este

$$\begin{cases} x_{n+1} = F_1(x_n, y_n, z_n) \\ y_{n+1} = F_2(x_{n+1}, y_n, z_n) \\ z_{n+1} = F_3(x_{n+1}, y_{n+1}, z_n) \end{cases}, \quad n \in \mathbb{N}$$

obținând următorul algoritm.

Algoritmul metodei Gauss-Seidel

I. Date de intrare: coordonatele punctului inițial x_0, y_0, z_0 (date de tip double)
expresiile funcțiilor F_1, F_2, F_3 de trei variabile reale declarate ca date de tip double
$\varepsilon > 0$ eroarea admisă

II. Date de ieşire: n, numărul de iteraţii efectuate
x_n, y_n, z_n aproximaţia soluţiei

III. Paşii algoritmului

1. Calculează

$$\begin{cases} x_1 = F_1(x_0, y_0, z_0) \\ y_1 = F_2(x_0, y_0, z_0) \\ z_1 = F_3(x_0, y_0, z_0) \end{cases}$$

2. Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon \text{ sau } |y_n - y_{n-1}| \geq \varepsilon \text{ sau } |z_n - z_{n-1}| \geq \varepsilon$$

Calculează

$$\begin{cases} x_{n+1} = F_1(x_n, y_n, z_n) \\ y_{n+1} = F_2(x_{n+1}, y_n, z_n) \\ z_{n+1} = F_3(x_{n+1}, y_{n+1}, z_n) \end{cases}$$

3. Tipăreşte n, x_n, y_n, z_n; STOP.

Exemplu numeric:

Pentru sistemul

$$\begin{cases} 2x^2 - yz - 5x + 1 = 0 \\ y^2 - 2x - \ln z = 0 \\ z^2 - xy - 2z - 8 = 0 \end{cases}$$

adus în forma echivalentă

$$\begin{cases} x = \sqrt{0.5(yz + 5x - 1)} \\ y = \sqrt{2x + \ln z} \\ z = \sqrt{xy + 2z + 8} \end{cases}$$

se alege punctul inițial $(x_0, y_0, z_0) = (10, 10, 10)$ se construiește șirul iterativ

$$\begin{cases} x_{n+1} = \sqrt{0.5(y_n z_n + 5x_n - 1)} \\ y_{n+1} = \sqrt{2x_{n+1} + \ln z_n} \\ z_{n+1} = \sqrt{x_{n+1} y_{n+1} + 2z_n + 8} \end{cases}, \quad n \in \mathbb{N}.$$

Să se aplice algoritmul Gauss-Seidel considerând $\varepsilon = 0.001$.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Se construiește clasa SistemNeliniar, clasă ce reprezintă clasa de bază pentru metodele iterative de rezolvare a sistemelor neliniare. Aceasta conține elementele comune tuturor algoritmilor de acest tip care vor urma:

- numărul de ecuații și necunoscute ale sistemului (k);
- funcțiile sistemului (F);
- vectorii aproximațiilor succesive (Xn, Xn_1);
- numărul de iterații (n);
- precizia de calcul ε;
- aproximațiile inițiale ($x0$, $y0$, $z0$);
- aproximațiile curente și precedente (xn, yn, zn, respectiv xn_1, yn_1, zn_1);
- constructorul pentru inițializarea datelor comune;
- metoda abstractă Rezolvare(), care se implementează în clasele derivate.

Constructorul inițializează numărul de ecuații al sistemului, funcțiile care definesc sistemul, vectorii necesari aproximărilor și precizia utilizată în algoritm.

Metoda Rezolvare() este declarată abstract, urmând ca fiecare metodă numerică derivată să implementeze propriul algoritm de rezolvare.

Algorithm 18 Clasa SistemNeliniar

```
public abstract class SistemNeliniar
{
    protected int k, n;
    protected Func<decimal[], decimal>[] F;
    protected decimal[] Xn, Xn_1;
    protected decimal epsilon;
    protected decimal x0, y0, z0;
    protected decimal xn, yn, zn;
    protected decimal xn_1, yn_1, zn_1;
    protected SistemNeliniar(int k, Func<decimal[], decimal>[] F,
        decimal epsilon)
    {
        this.k = k;
        this.F = F;
        Xn = new decimal[k];
        Xn_1 = new decimal[k];
        this.epsilon = epsilon;
    }
    public abstract void Rezolvare();
}
```

În cazul metodei aproximațiilor succesive pentru sisteme de ecuații neliniare, folosim clasa MetodaAproximatiilorSuccesive ce moștenește clasa SistemNeliniar și implementează metoda aproximațiilor succesive pentru rezolvarea unui sistem de două ecuații neliniare.

Constructorul acestei clase primește funcțiile de iterație pentru necunoscutele x și y, precizia dorită ε și aproximațiile inițiale ale soluției $x0$, respectiv $y0$. Observăm că dorim un vector de funcții F pentru clasa de bază, de aceea la apelul acesteia, inițializăm un nou vector care conține cele două funcții.

Metoda Exemple() va conține exemple de sisteme neliniare și ilustrează modul de utilizare al algoritmului pentru determinarea soluției aproximative.

Pentru metoda Gauss-Seidel pentru sisteme de ecuații neliniare vom folosi clasa MetodaGaussSeidel. Aceasta moștenește clasa SistemNeliniar și implementează metoda Gauss-Seidel pentru rezolvarea iterativă a unui sistem de trei ecuații neliniare. Constructorul primește funcțiile de iterație pentru necunoscutele x, y și z, precizia dorită și aproximațiile inițiale ale soluției.

Algorithm 19 Clasa MetodaAproximațiilorSuccesive

```
public class MetodaAproximatiilorSuccesive :  SistemNeliniar
{
    public MetodaAproximatiilorSuccesive(Func<decimal[], decimal> f,
        Func<decimal[], decimal> g, decimal epsilon, decimal x0,
        decimal y0) :  base(2, new[]  f, g , epsilon)
    {
        this.x0 = x0;
        this.y0 = y0;
    }
    public static void Exemple()
    {
        // Exemple
    }
    public static void Calculare()
    {
        // Algoritm
    }
}
```

Algorithm 20 Clasa MetodaGaussSeidel

```
public class MetodaGaussSeidel :  SistemNeliniar
{
    public MetodaGaussSeidel(Func<decimal[], decimal> f1,
        Func<decimal[], decimal> f2, Func<decimal[], decimal> f3,
        decimal epsilon, decimal x0, decimal y0, decimal z0)
        :  base(3, new[]  f1, f2, f3 , epsilon)
    {
        this.x0 = x0;
        this.y0 = y0;
        this.z0 = z0;
    }
    public static void Exemple()
    {
        // Exemple
    }
    public override void Rezolvare()
    {
        // Algoritm
    }
}
```

Capitolul 7

Laboratorul 7. Sisteme neliniare. Metoda lui Newton

METODA GAUSS-SEIDEL PENTRU SISTEME DE ECUAȚII NELINIARE

(două ecuații)

Observație: În cazul sistemelor de două ecuații neliniare

$$\begin{cases} x = F(x,y) \\ y = G(x,y) \end{cases}$$

șirul iterativ al metodei Gauss-Seidel va fi

$$\begin{cases} x_{n+1} = F(x_n, y_n) \\ y_{n+1} = G(x_{n+1}, y_n) \end{cases}, \quad n \in \mathbb{N}$$

iar algoritmul este:

I. Date de intrare:

x_0 şi y_0 iteraţia iniţială (date de tip double)

F şi G funcţii de două variabile de tip double

$\varepsilon > 0$ eroarea admisă (dată de tip double)

II. Date de ieşire: n, numărul de iteraţii efectuate

x_n şi y_n ultima iterație ce aproximează soluţia cu eroarea admisă

III. Paşii algoritmului

1. Calculează

$$\begin{cases} x_1 := F(x_0, y_0) \\ y_1 := G(x_0, y_0), \end{cases}$$

2. Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon \quad \text{sau} \quad |y_n - y_{n-1}| \geq \varepsilon$$

Calculează

$$\begin{cases} x_{n+1} := F(x_n, y_n) \\ y_{n+1} := G(x_{n+1}, y_n), \end{cases}$$

3. Tipăreşte n, x_n, y_n. STOP.

Exemplu numeric:
Sistemul neliniar simetric

$$\begin{cases} x^2 + y^2 = 5 \\ xy = 2 \end{cases}$$

are soluţiile $(-2,-1), (-1,-2), (1,2), (2,1)$ şi în vecinătatea punctului $(2,1)$ se scrie sub forma

$$\begin{cases} x = \sqrt{5-y^2} = F(x,y) \\ y = \frac{2}{x} = G(x,y) \end{cases}.$$

Să se aplice algoritmul lui Gauss-Seidel considerând punctul inițial $(x_0, y_0) = (1.5, 1.5)$ și $\varepsilon = 10^{-4}$ și să se compare numărul de iterații obținute cu cel furnizat de metoda clasică a aproximațiilor succesive.

METODA LUI NEWTON PENTRU SISTEME DE ECUAȚII NELINIARE

Considerăm sistemul de ecuații neliniare

$$\begin{cases} F(x,y) = 0 \\ G(x,y) = 0 \end{cases}$$

și aproximăm soluția acestui sistem utilizând metoda iterativă a lui Newton.

Algoritmul metodei lui Newton pentru sisteme neliniare

I. Date de intrare: coordonatele punctului iniţial x_0, y_0
expresia funcţiei F, expresia funcţiei G
expresiile funcţiilor $dfx = \frac{\partial F}{\partial x}$, $dfy = \frac{\partial F}{\partial y}$, $dgx = \frac{\partial G}{\partial x}$, $dgy = \frac{\partial G}{\partial y}$ (toate cele 6 funcții sunt de două variabile reale)
$\varepsilon > 0$ eroarea admisă

II. Date de ieşire: n, numărul de iteraţii efectuate
x_n, y_n aproximaţia soluţiei

III. Paşii algoritmului

1. Se citesc x_0, y_0, ε
2. Se definesc funcţiile F, G, $\frac{\partial F}{\partial x}$, $\frac{\partial F}{\partial y}$, $\frac{\partial G}{\partial x}$, $\frac{\partial G}{\partial y}$
3. Calculează

$$J(x_0, y_0) := \frac{\partial F}{\partial x}(x_0, y_0) \cdot \frac{\partial G}{\partial y}(x_0, y_0) - \frac{\partial F}{\partial y}(x_0, y_0) \cdot \frac{\partial G}{\partial x}(x_0, y_0)$$

Calculează

$$x_1 := x_0 - \left(\frac{1}{J(x_0, y_0)}\right) \cdot \left[F(x_0, y_0) \cdot \frac{\partial G}{\partial y}(x_0, y_0) - G(x_0, y_0) \cdot \frac{\partial F}{\partial y}(x_0, y_0)\right]$$

$$y_1 := y_0 - \left(\frac{1}{J(x_0, y_0)}\right) \cdot [\frac{\partial F}{\partial x}(x_0, y_0) \cdot G(x_0, y_0) - \frac{\partial G}{\partial x}(x_0, y_0) \cdot F(x_0, y_0)]$$

4. Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon \quad \text{sau} \quad |y_n - y_{n-1}| \geq \varepsilon$$

Calculează

$$J(x_n, y_n) := \frac{\partial F}{\partial x}(x_n, y_n) \cdot \frac{\partial G}{\partial y}(x_n, y_n) - \frac{\partial F}{\partial y}(x_n, y_n) \cdot \frac{\partial G}{\partial x}(x_n, y_n)$$

$$x_{n+1} := x_n - \left(\frac{1}{J(x_n, y_n)}\right) \cdot [F(x_n, y_n) \cdot \frac{\partial G}{\partial y}(x_n, y_n) - G(x_n, y_n) \cdot \frac{\partial F}{\partial y}(x_n, y_n)]$$

$$y_{n+1} := y_n - \left(\frac{1}{J(x_n, y_n)}\right) \cdot [\frac{\partial F}{\partial x}(x_n, y_n) \cdot G(x_n, y_n) - \frac{\partial G}{\partial x}(x_n, y_n) \cdot F(x_n, y_n)]$$

5. Tipăreşte n, x_n, y_n. STOP.

Exemple numerice:

1. Cu metoda lui Newton să se găsească soluția aproximativă a sistemului

$$\begin{cases} 2x^3 - y^2 - 1 = 0 \\ xy^3 - y - 4 = 0 \end{cases}$$

pornind de la punctul inițial $(x_0, y_0) = (1.2, 1.7)$ cu $\varepsilon = 10^{-4}$. Se va considera $F(x, y) = 2x^3 - y^2 - 1$, $G(x, y) = xy^3 - y - 4$, $\frac{\partial F}{\partial x}(x, y) = 6x^2$, $\frac{\partial F}{\partial y}(x, y) = -2y$, $\frac{\partial G}{\partial x}(x, y) = y^3$ și $\frac{\partial G}{\partial y}(x, y) = 3xy^2 - 1$.

2. Cu metoda lui Newton să se găsească soluția aproximativă a sistemului

$$\begin{cases} x^2 + y^2 = 10 \\ \sqrt{x + y} = 2 \end{cases}.$$

pornind odată de la punctul inițial $(x_0, y_0) = (0.9, 3.1)$ și apoi de la punctul inițial $(x_0, y_0) = (2.9, 1.1)$ cu $\varepsilon = 10^{-4}$ și să se compare rezultatul obținut cu soluția exactă $(1, 3)$, respectiv $(3, 1)$. Se va considera $F(x, y) = x^2 + y^2 - 10$, $G(x, y) = \sqrt{x + y} - 2$, $\frac{\partial F}{\partial x}(x, y) = 2x$, $\frac{\partial F}{\partial y}(x, y) = 2y$, $\frac{\partial G}{\partial x}(x, y) = \frac{1}{2\sqrt{x+y}}$ și $\frac{\partial G}{\partial y}(x, y) = \frac{1}{2\sqrt{x+y}}$.

Observație: Pentru implementarea metodei modificate a lui Newton se modifică pasul 4. în algoritmul de mai sus astfel:

Pasul 4': Pornind cu $n \geq 1$, cât timp

$$|x_n - x_{n-1}| \geq \varepsilon \quad \text{sau} \quad |y_n - y_{n-1}| \geq \varepsilon$$

Calculează

$$x_{n+1} := x_n - \left(\frac{1}{J(x_0, y_0)}\right) \cdot [F(x_n, y_n) \cdot \frac{\partial G}{\partial y}(x_n, y_n) - G(x_n, y_n) \cdot \frac{\partial F}{\partial y}(x_n, y_n)]$$

$$y_{n+1} := y_n - \left(\frac{1}{J(x_0, y_0)}\right) \cdot [\frac{\partial F}{\partial x}(x_n, y_n) \cdot G(x_n, y_n) - \frac{\partial G}{\partial x}(x_n, y_n) \cdot F(x_n, y_n)]$$

unde

$$J(x_0, y_0) := \frac{\partial F}{\partial x}(x_0, y_0) \cdot \frac{\partial G}{\partial y}(x_0, y_0) - \frac{\partial F}{\partial y}(x_0, y_0) \cdot \frac{\partial G}{\partial x}(x_0, y_0)$$

Exemplu: Să se implementeze metoda modificată a lui Newton pentru sistemul

$$\begin{cases} x^2 + y^2 = 10 \\ \sqrt{x + y} = 2 \end{cases}$$

de la exemplul al doilea și să se compare rezultatul și numărul de iterații făcute cu cel de la metoda lui Newton.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru rezolvarea sistemelor de ecuații neliniare cu metoda lui Newton construim clasa MetodaNewton ce moștenește clasa SistemNeliniar. În cadrul algoritmului sunt utilizate funcțiile care definesc sistemul și derivatele lor parțiale pentru calculul aproximațiilor succesive ale soluției.

Constructorul clasei MetodaNewton primește ca argumente funcțiile care definesc sistemul, derivatele parțiale necesare construirii matricei Jacobiene, precizia de calcul și aproximațiile inițiale ale necunoscutelor.

În metoda Rezolvare(), algoritmul pornește de la aproximațiile inițiale x_0 și y_0. La fiecare iterație sunt evaluate funcțiile sistemului de ecuații și derivatele parțiale ale funcțiilor, apoi se calculează determinantul matricei Jacobiene. Folosind relațiile metodei lui Newton, sunt obținute noile aproximații ale necunoscutelor. Procesul continuă până când diferențele dintre două aproximații succesive devin mai mici decât precizia ε și soluția finală este memorată în vectorul Xn.

Algorithm 21 Clasa MetodaNewton

```
public class MetodaNewton :  SistemNeliniar
{
    protected Func<decimal[], decimal> f, g;
    protected Func<decimal[], decimal> dfx, dgx, dfy, dgy;
    public MetodaNewton(Func<decimal[], decimal> f,
        Func<decimal[], decimal> g, Func<decimal[], decimal> dfx,
        Func<decimal[], decimal> dgx, Func<decimal[], decimal> dfy,
        Func<decimal[], decimal> dgy, decimal epsilon, decimal x0,
        decimal y0) :  base(2, new[]  f, g , epsilon)
    {
        this.f = f;
        this.g = g;
        this.dfx = dfx;
        this.dgx = dgx;
        this.dfy = dfy;
        this.dgy = dgy;
        this.x0 = x0;
        this.y0 = y0;
    }
    public static void Exemple()
    {
        // Exemple
    }
    public override void Rezolvare()
    {
        // Algoritm
    }
```

Capitolul 8

Laboratorul 8. Polinomul Taylor de aproximare locală

POLINOMUL LUI TAYLOR

Polinomul lui Taylor în vecinătatea $[a-h, a+h]$ a punctului $x = a$ este,

$$T_n f(x) = \sum_{k=0}^{n} \frac{(x-a)^k}{k!} \cdot f^{(k)}(a) \tag{8.1}$$

iar estimarea erorii de aproximare a funcției f va fi

$$|R_n f(x)| \leq \frac{1}{(n+1)!} \cdot |x-a|^{n+1} \cdot \left\| f^{(n+1)} \right\|_C \qquad \text{dacă} \quad f \in C^{n+1}[a-h, a+h].$$

Din formula (8.1) se observă că polinomul lui Taylor poate fi obținut recurent prin

$$T_0 f(x) = f(a)$$

$$T_k f(x) = T_{k-1} f(x) + \frac{(x-a)^k}{k!} \cdot f^{(k)}(a), \qquad \text{pentru} \quad k = \overline{1, n}.$$

Astfel, se obține algoritmul următor :

Algoritm pentru polinomul lui Taylor

I. Date de intrare:

n gradul polinomului de interpolare

a nodul de interpolare

$f^{(k)}(a), \quad k = \overline{0, n}$, valorile pe nodul a

$u \in [a, b]$ punctul în care se aproximează valoarea funcției f

II. Date de ieșire:

$T_n f(u)$ aproximația lui $f(u)$ prin polinomul lui Taylor

III. Pașii algoritmului:

1. fie $f_0 := f(a)$, $p(0) := 1$ și pentru $k = \overline{1,n}$

$$f_k := f^{(k)}(a)$$

$$p(k) := p(k-1) \cdot k$$

2. fie $T_0 f(u) = f(a)$ și pentru $k = \overline{1,n}$

$$T_k f(u) = T_{k-1} f(u) + \frac{1}{p(k)} \cdot (u-a)^k \cdot f_k$$

3. Tipărește $T_n f(u)$. STOP.

Exemple numerice:

1. Pentru $n = 7, a = 0, u = 1,\ f^{(k)}(a) = 1, \quad k = \overline{0,n}$ să se calculeze $T_7 f(u)$ și să se compare rezultatul cu e. Se obtine aici o aproximare a numărului e folosind polinomul lui Taylor pentru interpolarea funcției $f(x) = e^x$.

2. Pentru $n = 7,\ u = \frac{\pi}{4},\ a = 0, f^{(0)}(a) = 0, f^{(1)}(a) = 1, f^{(2)}(a) = 0, f^{(3)}(a) = -1, f^{(4)}(a) = 0, f^{(5)}(a) = 1, f^{(6)}(a) = 0, f^{(7)}(a) = -1$ să se calculeze $T_7 f(u)$ și să se compare rezultatul cu $\sin\frac{\pi}{4} = \frac{\sqrt{2}}{2}$. Se obține o aproximare a numărului $\sqrt{2}$ folosind polinomul lui Taylor pentru interpolarea funcției $f(x) = \sin x$. Să se aplice apoi algoritmul pentru $u = \frac{\pi}{18} = 10^0$ aproximând astfel numărul $\sin(10^0)$.

3. Să se construiască o problemă similară pentru $f(x) = \cos x$, $n = 8$ si $u = \frac{\pi}{6}$ cu $a = 0, f^{(0)}(a) = 1,\ f^{(1)}(a) = 0, f^{(2)}(a) = -1, f^{(3)}(a) = 0,\ f^{(4)}(a) = 1, f^{(5)}(a) = 0, f^{(6)}(a) = -1, f^{(7)}(a) = 0, f^{(8)}(a) = 1$ (comparând rezultatul obținut cu $\cos\frac{\pi}{6} = \frac{\sqrt{3}}{2}$ se va obține o aproximație pentru $\sqrt{3}$).

Observație: Se poate construi și algoritmul pentru trasarea graficului polinomului de aproximare locală Taylor.

Algoritm pentru polinomul lui Taylor (cu trasarea graficului)

I. Date de intrare:

n gradul polinomului de interpolare

a nodul de interpolare, b capătul drept al intervalului

$f[k], \quad k = \overline{0,n},$ valorile derivatei pe nodul a

$u[j] \in [a,b],\ j = \overline{0,1000}:$ punctele în care se aproximează valoarea funcției f

II. Date de ieșire:

$T[n,j],\ j = \overline{0,1000}$ aproximația lui $f(u[j])$ prin polinomul lui Taylor

III. Pașii algoritmului:

1. fie $p[0] := 1,\ h = \frac{b-a}{1000}$

2. Pentru $j = \overline{0,1000}$ calculează $u[j] = a + j \cdot h$ și fie $T[0,j] = f[0]$

pentru $k = \overline{1,n}$

$$p[k] := p[k-1] \cdot k$$

$$T[k,j] = T[k-1,j] + \frac{1}{p[k]} \cdot (u[j]-a)^k \cdot f[k]$$

3. Tipărește (desenează punctul $(u[j], T[n,j])$, pentru $j = \overline{0,1000}$; STOP.

Observație: Se vor putea implementa polinoamele T1 (verde), T3 (roșu), și T5 (albastru), pentru sinus (negru), respectiv T2, T4, T6 pentru cosinus, precizând valoarea lui n în cazul fiecăruia la pașii 2 și 3 ai algoritmului.

Date numerice:1) Pentru $f(x) = \sin x$, $n = 7$, $a = 0$, $b = \pi$, $f[0] = 0$, $f[1] = 1$, $f[2] = 0$, $f[3] = -1$, $f[4] = 0$, $f[5] = 1$, $f[6] = 0$, $f[7] = -1$

2) Pentru $f(x) = \cos x$, $n = 6$, $a = 0$, $b = \pi$, $f[0] = 1$, $f[1] = 0$, $f[2] = -1$, $f[3] = 0$, $f[4] = 1$, $f[5] = 0$, $f[6] = -1$

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru următoarele laboratoare, vom avea prezentați atât algoritmii pentru obținerea valorilor numerice, cât și cei pentru trasarea graficului acestor valori. În acest îndrumător, vom acoperi explicațiile pentru algoritmele cu trasarea graficului. Tipul de proiect în care vom lucra este *Windows Forms App*.

Întâi, vom identifica noua clasă abstractă de care avem nevoie, în care vom adăuga proprietățile unui polinom: gradul polinomului n, nodurile de interpolare x, valorile funcției f pe aceste noduri y, valorile aproximate ale lui f, punctele în care se calculează aceste aproximații u, și o variabilă în care salvăm toate gradele precedent calculate pentru f, F, pe care propunem a fi creată sub forma unui vector de vectori. În constructor vom primi valorile lui n, x și y, și vom inițializa restul vectorilor.

Tot în această clasă, vom avea o valoare constantă ce ne va spune în câte puncte dorim să calculăm valorile lui f. Pentru a ne asigura că graficul nu are goluri (cu posibile excepții), vom da valoarea 1000. Vom redenumi metoda *Rezolvare* la *CalculeazaPuncte* pentru că s-a schimbat rolul acesteia.

Haideți să discutăm despre cum vom vizualiza elementele în partea de *Designer*: vom adăuga un *pictureBox* pentru care vom desena graficul funcției aproximate și prin care îl vom afișa, iar pentru fiecare exemplu de la fiecare laborator, vom adăuga câte un *buton*.

Vom crea o metodă pentru desenarea schiței graficului în clasa *Form1*. Scopul acesteia este pentru a afișa valorile pe axele Ox și Oy, pentru a putea observa scalarea graficului. Vom păstra 40 de pixeli în stânga și 20 de pixeli jos, iar pentru a păstra valorile acestea mai ușor, vom declara variabilele *latime* și *inaltime*. De asemenea, vom păstra valoarea intersecției axelor ca și un punct O reprezentând originea, iar pentru că in grafica unui calculator, valoarea lui y scade atunci când urcăm, valoarea y a originii va avea atribuită valoarea salvată în *inaltime*.

Avem nevoie și de o variabilă *imagine*, inițializată la începutul metodei *DeseneazaSchita* cu lățimea și înălțimea *picturebox*-ului, și de una *graphica*, ce va face modificări în *imagine*. Desenăm întâi liniile ce vor reprezenta axele de coordonate folosind metoda *DrawLine* a obiectului *graphica*: pentru axa Ox, valorile lui x sunt 0 și lățimea *picturebox*-ului, cu ambele valori y fiind valoarea Y a *originii*, iar pentru axa Oy, valorile lui y

Algorithm 22 Clasa de bază Polinom

```
1: public abstract class Polinom
2: {
3:     public const int nrPuncte = 1000;
4:     public int n;
5:     public decimal[] x;
6:     public decimal[] y;
7:     public decimal[] u;
8:     public decimal[] f;
9:     public decimal[][] F;
10:    public Sistem(int n, decimal[] x, decimal[] y)
11:    {
12:        this.n = n;
13:        this.x = x;
14:        this.y = y;
15:        u = new decimal[nrPuncte];
16:        f = new decimal[nrPuncte];
17:        F = new decimal[n + 1][];
18:    }
19:    public abstract void CalcularePuncte();
20: }
```

sunt 0 și înălțimea *picturebox*-ului, cu ambele valori x fiind valoarea X a *originii*. Pentru afișarea valorilor ce ne vor indica scalarea graficului, avem nevoie de variabilele *x0, xn, y0* și *yn*, care deocadată nu sunt inițializate. Vom folosi metoda *DrawString* a obiectului *graphica* pentru fiecare dintre aceste patru valori. Vom crea și folosi și alte două variabile *font* și *pensula*, iar pozițiile fiecărei valori sunt următoarele: pentru *y0*, 0 și *inaltime* - 20, pentru *yn*, 0 și 0, pentru *x0*, valorile X și Y ale *originii*, iar pentru *xn*, *latime* și valoarea Y a *originii*. La final, atribuim proprietății *Image* a *picturebox*-ului valoarea *imagine*.

Acum tot ce mai rămâne de pregătit este metoda *DeseneazaGrafic*. Vom primi parametrul *polinom* de unde extragem valorile pentru *x0* și *xn*. Pentru valorile *y0* și *yn*, scopul nu este să salvăm prima și ultima valoare din graficul lui f, ci cea mai mică și cea mai mare valoare din întreg graficul. Și pentru că se dorește afișarea graficelor mai multor grade ale polinomului, dorim să luăm cea mai mică și cea mai mare valoare din toate valorile salvate în F. Pentru a transforma acest vector de vectori într-un vector unidimensional, se poate folosi metoda *SelectMany*. După care putem folosi metodele *Min* și *Max*.

Vom avea nevoie de o metodă care transformă valorile calculate în valorile ce trebuiesc afișate, numită *TransformaInPunct*. Pentru calculele din această metodă, avem nevoie de două variabile *scalareX* și *scalareY*, care sunt pur și simplu diferența dintre *xn* și *x0*, respectiv dintre *yn* și *y0*. Haideți să de gândim cum ajungem la formula pentru această calculare: considerând că am avea *x0=0* și originea nu ar exista, am dori să transformăm un x din intervalul *(0, xn)* la o valoare mărită pe toti pixelii din *pictureBox*. Asta ar

Algorithm 23 Metoda DeseneazaSchita

```
1: public partial class Form1 :  Form
2: {
3:     public Graphics graphica;
4:     public Bitmap imagine;
5:     public Pen gri = new Pen(Color.Gray, 3);
6:     public PointF O;
7:     public decimal x0, y0, xn, yn;
8:     public float latime, inaltime;
9:     public Form1()
10:    {
11:        InitializeComponent();
12:        latime = pictureBox1.Width - 40;
13:        inaltime = pictureBox1.Height - 20;
14:        O = new PointF(40, inaltime);
15:    }
16:    public void DeseneazaSchita()
17:    {
18:        imagine = new Bitmap(pictureBox1.Width, pictureBox1.Height);
19:        graphica = Graphics.FromImage(imagine);
20:
21:        graphica.DrawLine(gri, 0, O.Y, pictureBox1.Width, O.Y);
22:        graphica.DrawLine(gri, O.X, 0, O.X, pictureBox1.Height);
23:        Font font = new Font("Arial", 12, FontStyle.Bold);
24:        SolidBrush pensula = new SolidBrush(Color.Black);
25:
26:        graphica.DrawString(y0, font, pensula, 0, inaltime - 20);
27:        graphica.DrawString(yn, font, pensula, 0, 0);
28:        graphica.DrawString(x0, font, pensula, O.X, O.Y);
29:        graphica.DrawString(xn, font, pensula, latime, O.Y);
30:        pictureBox1.Image = imagine;
31:    }
32: }
```

însemna că înmulțim x cu lățimea, și împarțim la *xn*: `x * pictureBox.Width / xn`. Pentru y ar fi similar, doar că folosim înălțimea, și valoarea calculată va fi scăzută din înalțimea totală (datorită faptului că la grafica unui calculator, valorile lui y pornesc de sus și merg în jos pe cum y crește). Dacă intervalul are capătul stâng *x0*, atunci trebuie întâi să scădem această valoare din x, și nu vom împărți la *xn*, ci la *scalareX*: `(x-x0) * pictureBox.Width / scalareX`. Dacă mai introducem și ideea de *origine*, în loc de întreaga lățime, folosim variabila *latime*, și trebuie să pornim de la valoarea originii: `O.X + (x-x0) * latime / scalareX`. Pentru y, în loc să scădem din întreaga înălțime, scădem din *O.Y*, iar restul modificărilor sunt similare ca și pentru x.

Acum că avem valorile corecte, mai trebuie doar să le afișăm. Parcurgem întregul șir de numere *x*, transformăm în punct, și folosim metoda *DrawEllipse* a obiectului *grafica*. Acesta necesită și definirea chenarului în care se va afișa punctul, așa că luăm lățimea creionului primit ca parametru, și scădem jumătate din aceasta din valoarea punctului calculat, pentru a centra punctul afișat.

Algorithm 24 Metoda DeseneazaGrafic

```
1: public Polinom polinom;
2: public decimal scalareX, scalareY;
3: public void DeseneazaGrafic(Pen creion, decimal[] x, decimal[] y)
4: {
5:     decimal[] valori = polinom.F.SelectMany(v => v).ToArray();
6:     x0 = x.First();
7:     xn = x.Last();
8:     y0 = valori.Min();
9:     yn = valori.Max();
10:    scalareX = xn - x0;
11:    scalareY = yn - y0;
12:
13:    DeseneazaSchita();
14:    for (int i = 0; i < x.Length; i++)
15:    {
16:        Point punct = TransformaInPunct(x[i], y[i]);
17:        float w = creion.Width;
18:        graphica.DrawEllipse(creion, punct.X-w/2, punct.Y-w/2, w, w);
19:    }
20:    pictureBox1.Image = imagine;
21: }
22: public PointF TransformaInPunct(decimal x, decimal y)
23: {
24:    float X = O.X + (float)((x - x0) * (decimal)latime / scalareX);
25:    float Y = O.Y - (float)((y - y0) * (decimal)inaltime / scalareY);
26:    return new PointF(X, Y);
27: }
```

Acum că am creat metodele de bază pentru a putea afișa un grafic, trecem la implementarea clasei *PolinomulLuiTaylor*: constructorul primește aceiași parametri ca și clasa *Polinom*, apelează constructorul clasei de bază, și suprascriem metoda *Calculare Puncte*, unde se va scrie algoritmul descris mai sus.

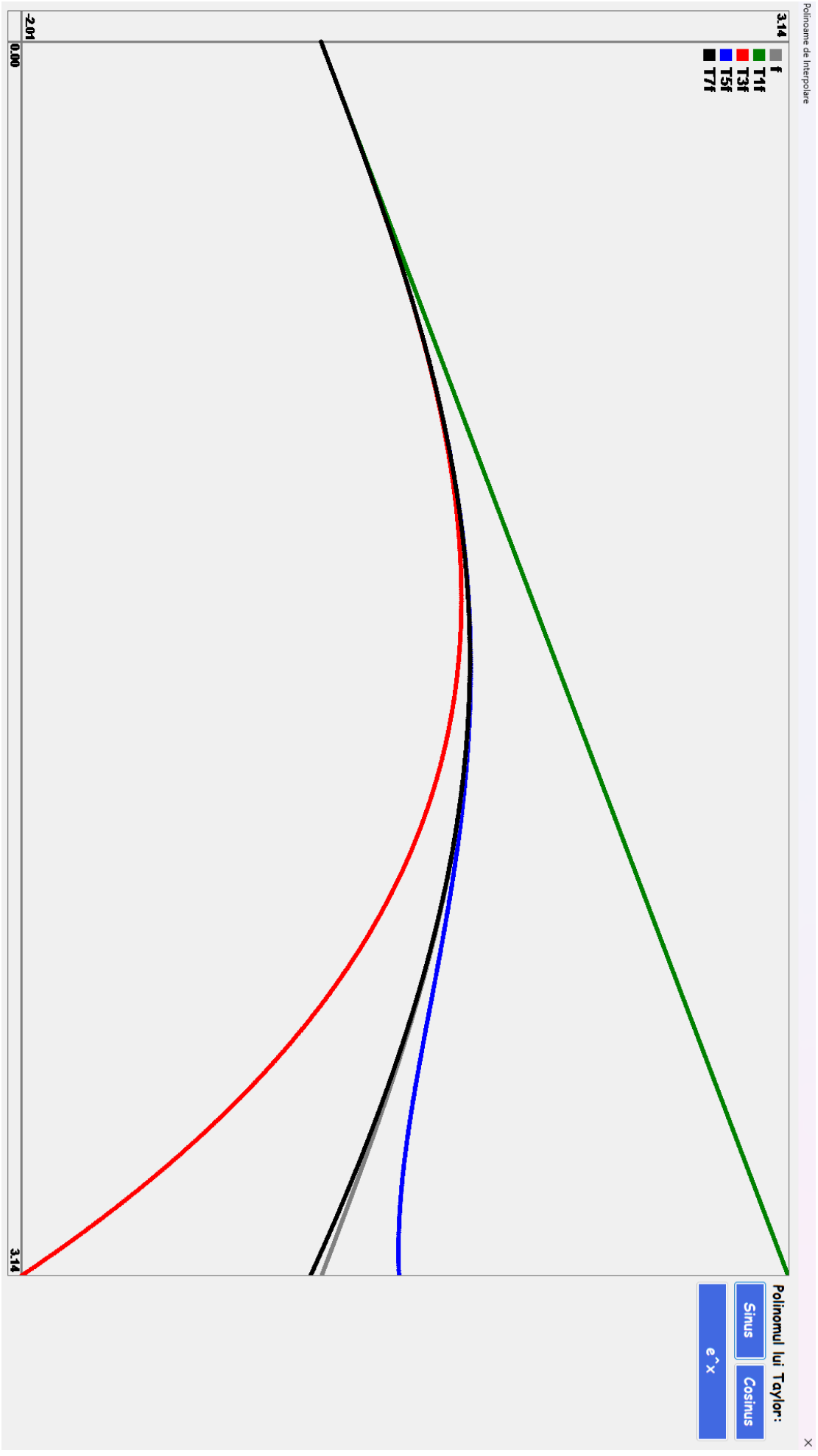
Algorithm 25 Clasa PolinomulLuiTaylor

```
1: public class PolinomulLuiTaylor :  Polinom
2: {
3:     public PolinomulLuiTaylor(int n, decimal[] x, decimal[] y)
4:         :  base(n, x, y) { }
5:     public override void Calculare Puncte()
6:     {
7:         // Algoritm
8:     }
9: }
```

Pentru implementarea exemplului, vom da valoare proprietății *polinom*, gradul acestuia având valoarea lui n, pentru x atribuim valorile lui a și b, iar pentru y toate valorile lui f primite. Apelăm metoda *Calculare Puncte*, apoi metoda *DeseneazaGrafic* de câte ori este nevoie, cu culoarea aleasă, vectorul u și F de gradul pe care dorim să îl afișăm. Provocare: afișați și valoarea exactă a funcției cu gri, și afișați și o legendă a culorilor (pentru a ști gradul fiecărui polinom), așa cum puteti vedea în imaginea de mai jos.

Algorithm 26 Implementarea metodei click a butonului pentru afișarea exemplului

```
1: private void TaylorSinus_Click(object sender, EventArgs e)
2: {
3:     polinom = new PolinomulLuiTaylor(7, new[] {0M, (decimal)Math.PI},
4:         new[] {0M, 1, 0, -1, 0, 1, 0, -1});
5:     polinom.Calculare Puncte();
6:     DeseneazaGrafic(new Pen(Color.Green, 3), polinom.u, polinom.F[1]);
7:     DeseneazaGrafic(new Pen(Color.Red, 3), polinom.u, polinom.F[3]);
8:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.F[5]);
9:     DeseneazaGrafic(new Pen(Color.Black, 3), polinom.u, polinom.F[7]);
10: }
```


Polinoame de Interpolare
Polinomul lui Taylor:
Sinus
Cosinus
e^x
3.14
-2.01
0.00
3.14
f
T1f
T3f
T5f
T7f

Capitolul 9

Laboratorul 9. Polinomul de interpolare Lagrange

POLINOMUL DE INTERPOLARE AL LUI LAGRANGE. FORMULA LUI NEWTON

Polinomul de interpolare dat prin formula lui Newton este

$$N_m f(x) = f(x_0) + \sum_{i=1}^{m} (x - x_0) \cdot ... \cdot (x - x_{i-1}) \cdot [x_0, x_1, ..., x_i; f] = \tag{9.1}$$

$$= f(x_0) + \sum_{i=1}^{m} (x - x_0) \cdot ... \cdot (x - x_{i-1}) \cdot D^i f_0$$

iar restul dat cu ajutorul diferențelor divizate

$$R_m f(x) = u(x) \cdot [x, x_0, x_1, ..., x_m; f].$$

Se observă că în formula (9.1) s-au folosit doar diferențele divizate de pe latura superioară a tabloului de diferențe.

Observație: Din formula (9.1) rezultă că polinomul de interpolare al lui Newton verifică relația de recurență

$$N_i f(x; x_0, ..., x_i) = N_{i-1} f(x; x_0, ..., x_{i-1}) + (x - x_0) \cdot ... \cdot (x - x_{i-1}) \cdot [x_0, ..., x_i; f],$$

$\forall i = \overline{1, m}$, unde $N_0 f(x) = f(x_0)$. Această proprietate permite construirea recurentă a acestui polinom (deci programare mai facilă) precum și oprirea algoritmului odată atinsă precizia dorita, după ce în prealabil s-au reordonat nodurile în funcție de distanța lor până la punctul în care dorim să aproximăm funcția f. Astfel se obțin iterativ polinoamele lui Newton de gradele 1, 2, ...,m. Astfel, dacă

$$|N_i f(\alpha) - N_{i-1} f(\alpha)| < \varepsilon$$

atunci algoritmul se oprește la ordinul i, adică la polinomul $N_i f\left(x; x_0, ..., x_i\right)$. Însă gradul maxim al polinomului generat de algoritmul de mai jos este m.

Algoritmul polinomului lui Newton:

I. Date de intrare:

m gradul maxim al polinomului de interpolare (dată de tip int)

a_i, $i = \overline{0, m}$: nodurile de interpolare (șir de date de tip double)

f expresia funcției, sau valorile ei pe nodurile de interpolare $f\ (a_i)$, $i = \overline{0, m}$

ε eroarea absolută maximă admisibilă

$\alpha \in [a, b]$ punctul în care se aproximează valoarea funcției f

II. Date de ieșire:

$N_i f\left(\alpha\right)$ valoarea calculată cu precizia cerută ($i \leq m$ este gradul polinomului pentru care s-a atins această precizie)

sau un mesaj de eroare în care se specifică faptul că în condițiile date nu s-a putut calcula o aproximație cu precizia dorită.

III. Pașii algoritmului:

1. Se calculează diferențele divizate $[a_0, ..., a_i; f] = D^i f_0 = d[i, 0]$, $i = \overline{0, m}$, astfel

$$d[0, i] := f\left(a_i\right), \quad \text{pentru} \quad i = \overline{0, m}$$

Pentru $j = \overline{1, m}$

Pentru $i = \overline{0, m - j}$ calculează

$$d[j, i] := \frac{d[j-1, i+1] - d[j-1, i]}{a_{i+j} - a_i}$$

2. $i := 0$; $N_0 f\left(\alpha\right) = f\left(a_0\right)$; $p[0] := 1$

3. Începând cu $i \geq 1$ dacă $i \leq m$ atunci execută $p[i] := p[i-1] \cdot (\alpha - a_{i-1})$

$$N_i f\left(\alpha\right) := N_{i-1} f\left(\alpha\right) + p[i] \cdot d[i, 0]$$

dacă $\left|N_i f\left(\alpha\right) - N_{i-1} f\left(\alpha\right)\right| < \varepsilon$ atunci tipărește $N_i f\left(\alpha\right)$. STOP.

4. Dacă $\left|N_m f\left(\alpha\right) - N_{m-1} f\left(\alpha\right)\right| < \varepsilon$ atunci tipărește $N_m f\left(\alpha\right)$. STOP.

Altfel, tipărește " Nu s-a putut calcula o aproximație cu precizia dorită ". STOP.

Probleme propuse:

1. Valorile funcției $f\left(x\right) = \sqrt[3]{x}$ sunt date pe noduri în tabelul

x	1.0	1.1	1.3	1.5	1.6
$f\left(x\right)$	1	1.032	1.091	1.145	1.17

Folosind algoritmul lui Newton, sa se calculeze $f\left(1.15\right)$ cu o eroare absolută maximă de 10^{-2}.

2. Valorile unei funcții f sunt date pe noduri prin intermediul tabelului

x	1.00	1.08	1.13	1.20	1.27	1.31	1.38
$f\left(x\right)$	1.17520	1.30254	1.38631	1.50946	1.21730	1.22361	1.23470

Folosind algoritmul lui Newton, să se calculeze cu o precizie de 10^{-3} valorile funcției în punctele 1.134, 1.151, 1.185.

Polinomul lui Lagrange cu metoda lui Newton (trasarea graficului)

În calcule, la construcția formulei de interpolare a lui Newton pentru polinomul lui Lagrange, este util tabloul de diferențe divizate:

x	f	Df	D^2f	D^3f	.....	$D^{m-1}f$	D^mf
x_0	f_0	Df_0	D^2f_0	D^3f_0	.....	$D^{m-1}f_0$	D^mf_0
x_1	f_1	Df_1	D^2f_1	D^3f_1	.....	$D^{m-1}f_1$	
x_2	f_2	Df_2	D^2f_2	D^3f_2	.....		
....	....	.....	.....	.....			
x_{m-2}	f_{m-2}	Df_{m-2}	D^2f_{m-2}				
x_{m-1}	f_{m-1}	Df_{m-1}					
x_m	f_m						

unde $f_i = f(x_i), \quad i = \overline{0,m}$.

La construcția tabelului s-a folosit formula de recurență

$$D^k f(x_i) = \frac{D^{k-1}f(x_{i+1}) - D^{k-1}f(x_i)}{x_{i+k} - x_i}, \quad k = \overline{1,m}, \quad i = \overline{0,m-k}.$$

Algoritm pentru polinomul de interpolare al lui Lagrange cu metoda lui Newton (cu trasarea graficului)

I. Date de intrare:

m gradul maxim al polinomului de interpolare (dată de tip int)

$a[i]$, $i = \overline{0,m}$: nodurile de interpolare (șir de date de tip double)

f expresia funcției, sau valorile ei pe nodurile de interpolare $f[i]$, $i = \overline{0,m}$

$u[j] \in [a[0], a[m]]$, $j = \overline{0,1000}$, punctele în care se aproximează valorile funcției f

II. Date de ieșire: $N_m f(u[j])$, $j = \overline{0,1000}$, aproximațiile valorilor $f(u[j])$, $j = \overline{0,1000}$, prin polinomul lui Lagrange

III. Pașii algoritmului:

1. Se calculează diferențele divizate $D^i f_0 = d[i,0], \quad i = \overline{0,m}$, astfel

$$d[0,i] := f[i], \quad \text{pentru} \quad i = \overline{0,m}$$

Pentru $j = \overline{1,m}$

Pentru $i = \overline{0,m-j}$ calculează

$$d[j,i] := \frac{d[j-1,i+1] - d[j-1,i]}{a[i+j] - a[i]}$$

2. Calculează $h = \frac{a[m]-a[0]}{1000}$

Pentru $j = \overline{0,1000}$ calculează $u[j] := a[0] + j \cdot h$; $N[0,j] = f[0]$; $p[0] := 1$

Pentru $i = \overline{1,m}$ execută

$$p[i] := p[i-1] \cdot (u[j] - a[i-1])$$
$$N[i,j] := N[i-1,j] + p[i] \cdot d[i,0]$$

3. Pentru $j = \overline{0,1000}$ tipărește (desenează) punctul $(u[j], N[m,j])$. Stop.

Exemplu:
$m = 4$, se construiește polinomul de grad 4 care interpolează datele din tabelul:

x	1.0	1.1	1.3	1.5	1.6
$f(x)$	1	1.032	1.091	1.145	1.17

în vederea aproximării funcției $f(x) = \sqrt[3]{x}$.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

La bază, structura polinomului lui Newton este aceeași, deci nu este nevoie să adăugăm alte proprietăți pentru clasa *PolinomulLuiNewton*, dar va fi nevoie să adăugăm variabila d, pe care propunem să fie creată sub forma unui vector de vectori, precum F.

Algorithm 27 Clasa PolinomulLuiNewton

```
1: public class PolinomulLuiNewton :  Polinom
2: {
3:     public PolinomulLuiNewton(int n, decimal[] x, decimal[] y)
4:         :  base(n, x, y) { }
5:     public override void CalculeazaPuncte()
6:     {
7:         // Algoritm
8:     }
9: }
```

Implementarea exemplului este foarte similară cu cea a polinomului lui Taylor, doar că primim la fel de multe valori inițiale pentru x cât pentru y. Provocare: afișați și punctele primite inițial cu gri și mai mari pentru a putea fi observate, așa cum puteti vedea în imaginea de mai jos.

Algorithm 28 Implementarea metodei click a butonului pentru afișarea exemplului

```
1: private void NewtonRadicalOrd3_Click(object sender, EventArgs e)
2: {
3:     polinom = new PolinomulLuiNewton(4,
4:         new[] {1, 1.1M, 1.3M, 1.5M, 1.6M},
5:         new[] {1, 1.032M, 1.091M, 1.145M, 1.17M});
6:     polinom.CalculeazaPuncte();
7:     DeseneazaGrafic(new Pen(Color.Green, 3), polinom.u, polinom.F[1]);
8:     DeseneazaGrafic(new Pen(Color.Red, 3), polinom.u, polinom.F[2]);
9:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.F[3]);
10:    DeseneazaGrafic(new Pen(Color.Black, 3), polinom.u, polinom.F[4]);
11: }
```


Polinoame de Interpolare
1.19
1.00
1.00
1.60
f
N1f
N2f
N3f
N4f
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie

Capitolul 10

Laboratorul 10. Polinoame de interpolare Hermite şi polinomul Bernstein

POLINOMUL DE INTERPOLARE HERMITE CU DOUĂ NODURI DUBLE

Algoritmul polinomului de interpolare Hermite
I. Date de intrare
a, b : nodurile de interpolare (date de tip double)
fa, fb, dfa, dfb: valorile interpolate (date de tip double)
$x \in (a, b)$: punctul în care se realizează interpolarea (dată de tip double)
II. Date de ieșire: $P(x)$
III. Pași
1. Calculează

$$P(x) := \left(\left((b-x)^2 \cdot (x-a)\right) / (b-a)^2\right) \cdot dfa - \left(\left((x-a)^2 \cdot (b-x)\right) / (b-a)^2\right) \cdot dfb +$$
$$+ \left(\left((b-x)^2 \cdot (2 \cdot (x-a) + (b-a))\right) / (b-a)^3\right) \cdot fa +$$
$$+ \left(\left((x-a)^2 \cdot (2 \cdot (b-x) + (b-a))\right) / (b-a)^3\right) \cdot fb$$

2. Tipărește $P(x)$; Stop.

Exemple numerice:
1. $a = \frac{\pi}{6}$, $b = \frac{\pi}{3}$, $f(a) = \frac{1}{2}$, $f(b) = \frac{\sqrt{3}}{2}$, $df(a) = \frac{\sqrt{3}}{2}$, $df(b) = \frac{1}{2}$, iar pentru x se iau pe rând valorile $\frac{\pi}{4}$ și $\frac{\pi}{5}$. Se aproximează valorile funcției $\sin x$.
2. $a = 4$, $b = 9$, $f(a) = 2$, $f(b) = 3$, $df(a) = \frac{1}{4}$, $df(b) = \frac{1}{6}$, iar pentru x se iau pe rând valorile 5, 6, 7 și 8. Se aproximează valorile funcției $f(x) = \sqrt{x}$.

Algoritmul polinomului cubic de interpolare Hermite (cu trasarea graficului)

I. Date de intrare

a, b : nodurile de interpolare (date de tip double)

fa, fb, dfa, dfb: valorile interpolate (date de tip double)

II. Date de ieșire: graficul polinomului Hermite

III. Pași

1. Calculează $h = \frac{b-a}{1000}$

2. Pentru $j = \overline{0,1000}$ calculează $x[j] := a + j \cdot h$

$$P[j] := \left(\left((b - x[j])^2 \cdot (x[j] - a)\right) / (b-a)^2\right) \cdot dfa -$$
$$- \left(\left((x[j] - a)^2 \cdot (b - x[j])\right) / (b-a)^2\right) \cdot dfb +$$
$$+ \left(\left((b - x[j])^2 \cdot (2 \cdot (x[j] - a) + (b - a))\right) / (b-a)^3\right) \cdot fa +$$
$$+ \left(\left((x[j] - a)^2 \cdot (2 \cdot (b - x[j]) + (b - a))\right) / (b-a)^3\right) \cdot fb$$

3. Pentru $j = \overline{0,1000}$ tipărește (desenează) punctele $(x[j], P[j])$; Stop.

Exemple numerice:

1. Se dau $a = \frac{\pi}{6}$, $b = \frac{\pi}{2}$, $f(a) = \frac{1}{2}$, $f(b) = 1$, $df(a) = \frac{\sqrt{3}}{2}$, $df(b) = 0$. Se trasează graficul aproximației funcției $f(x) = \sin x$ pe intervalul $[\frac{\pi}{6}, \frac{\pi}{2}]$.

2. $a = 4$, $b = 9$, $f(a) = 2$, $f(b) = 3$, $df(a) = \frac{1}{4}$, $df(b) = \frac{1}{6}$, iar pentru x se iau pe rând valorile 5, 6, 7 și 8. Se trasează graficul aproximației funcției $f(x) = \sqrt{x}$.

POLINOMUL LUI BERNSTEIN

Polinomul lui Bernstein de grad m relativ la funcția f este definit prin formula:

$$(B_m f)(x) = \sum_{k=0}^{m} p_{m,k}(x) \cdot f\left(\frac{k}{m}\right), \quad \forall x \in [0,1]$$

unde

$$p_{m,k}(x) = C_m^k x^k (1-x)^{m-k} \geq 0, \quad \forall x \in [0,1], \quad \forall k = \overline{0,m}.$$

În programul de mai jos se implementează expresiile polinoamelor Bernstein de gradele 1 (culoarea negru), 2 (albastru), 3 (verde) și 4 (rosu) pentru aproximarea funcțiilor $f(x) = e^x$ (culoarea gri), (și eventual $g(x) = 1 + \sqrt{x}$), pe intervalul $[0,1]$.

Astfel, se vor calcula:

$$B_1(f)(x) = 1 - x + x \cdot e$$

$$B_2(f)(x) = (1-x)^2 + 2 \cdot x \cdot (1-x) \cdot \sqrt{e} + x^2 \cdot e$$

$$B_3(f)(x) = (1-x)^3 + 3 \cdot (1-x)^2 \cdot x \cdot pow(e, \frac{1}{3}) + 3 \cdot x^2 \cdot (1-x) \cdot pow(e, \frac{2}{3}) + x^3 \cdot e$$

$$B_4(f)(x) = (1-x)^4 + 4 \cdot (1-x)^3 \cdot pow(e, \frac{1}{4}) + 6 \cdot (1-x)^2 \cdot x^2 \cdot \sqrt{e} +$$

$$+4 \cdot (1-x) \cdot x^3 \cdot pow(e, \frac{3}{4}) + x^4 \cdot e$$

în punctele $x[j] := a + j \cdot h$ cu $h = \frac{1}{1000}$.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Polinomului lui Hermite are la bază două valori noi: derivata funcției în a și în b. Se pot adăuga două variabile diferite pentru acestea, dar cum ambii vectori x și y au tot numai două valori fiecare, se poate defini și un nou vector *df*, ce va fi primit ca parametru în constructor si atribuit proprietății cu același nume.

Algorithm 29 Clasa PolinomulHermite

```
1: public class PolinomulHermite :  Polinom
2: {
3:     public decimal[] df;
4:     public PolinomulHermite(int n, decimal[] x, decimal[] f,
5:         decimal[] df) :  base(n, x, f)
6:     {
7:         this.df = df;
8:     }
9:     public override void CalculeazaPuncte()
10:    {
11:        // Algoritm
12:    }
13: }
```

Pentru polinomul lui Bernstein, sunt două variante: folosiți formulele calculate pe hârtie pentru diferitele grade ale polinomului (așa cum a fost arătat mai sus), sau să se folosească formula generală pentru a putea atinge grade mai înalte ale polinomului (provocare). Pentru a doua variantă, avem nevoie ori de funcția de bază pe care o aproximăm, ori de un vector cu valorile necesare formulelor pentru fiecare grad, pentru funcția de bază. În codul de mai jos, am folosit prima variantă:

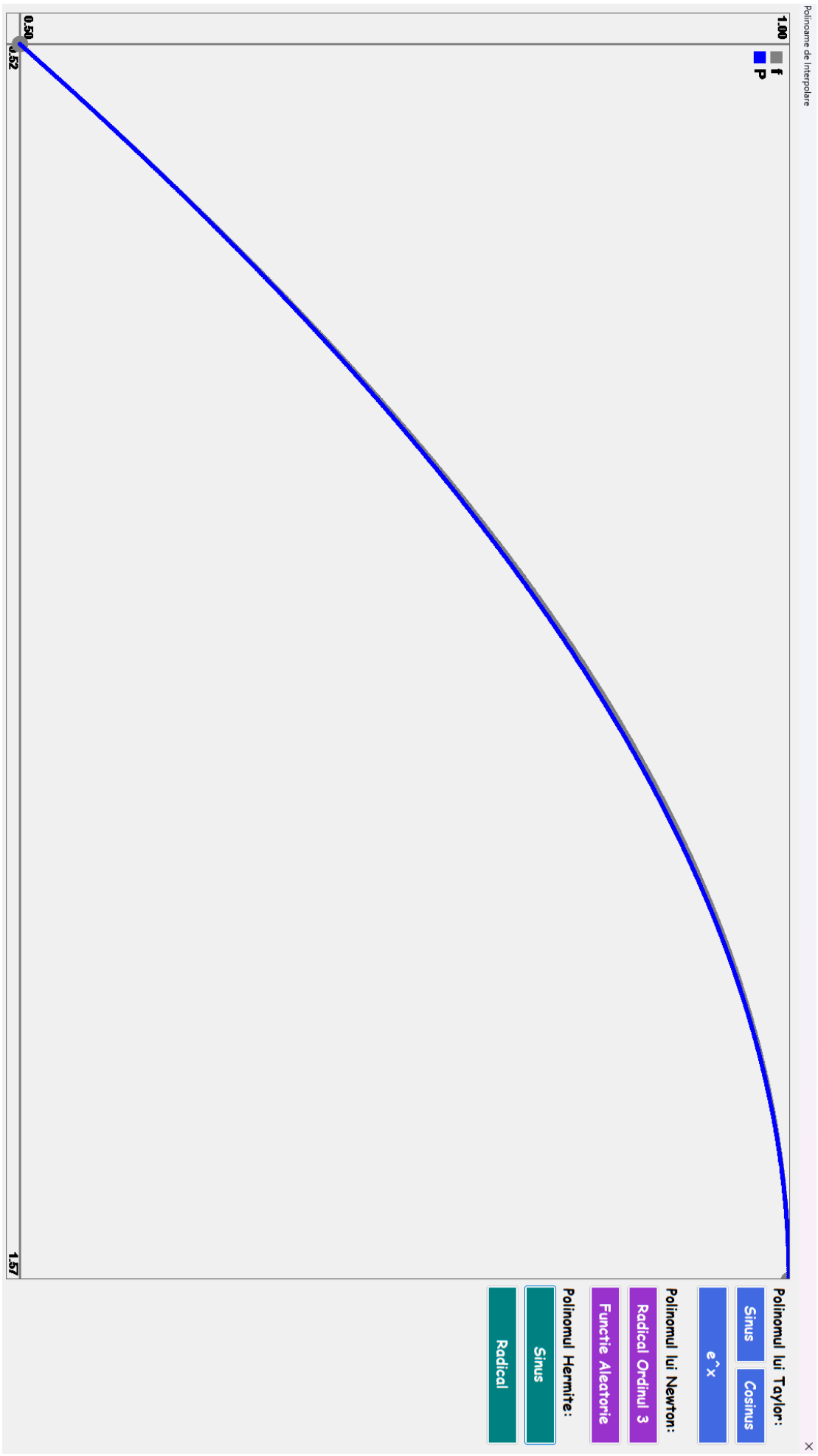
În implementarea exemplului polinomului Hermite, polinomul primește și valorile lui *df*, și avem doar un grafic de afișat înafară de valoarea exactă a funcției pe care o aproximăm. Cât pentru polinomul Bernstein, se primește de asemenea și funcția originală aproximată ca parametru.

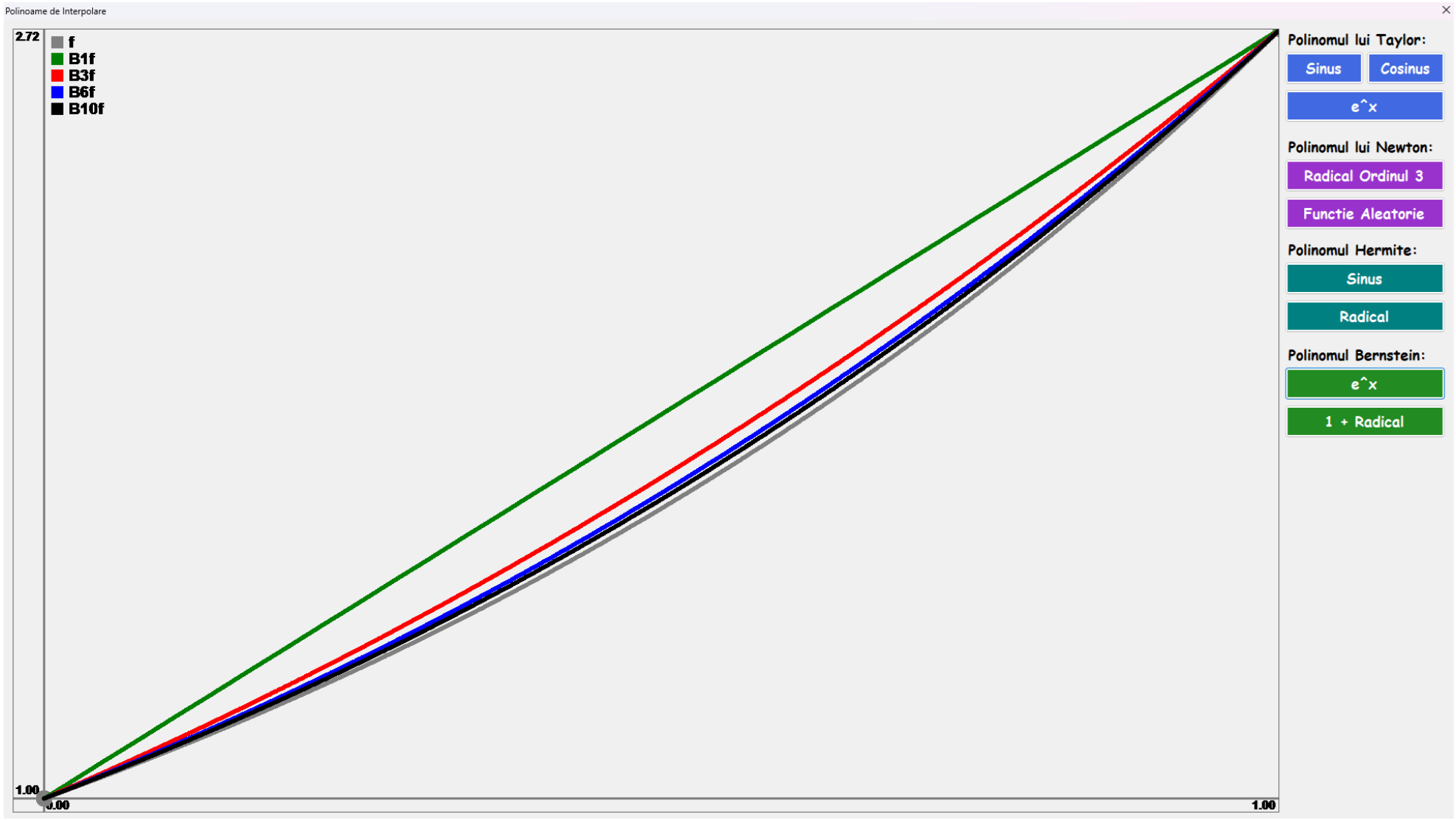
Algorithm 30 Clasa PolinomulBernstein

```
1: public class PolinomulBernstein :  Polinom
2: {
3:     public Func<decimal, decimal> func;
4:     public PolinomulBernstein(int n, decimal[] x, decimal[] y,
5:         Func<decimal, decimal> func) :  base(n, x, y)
6:     {
7:         this.func = func;
8:     }
9:     public override void CalculeazaPuncte()
10:    {
11:        // Algoritm
12:    }
13: }
```

Algorithm 31 Implementarea metodei click a butoanelor pentru afișarea exemplelor

```
1: private void HermiteSinus_Click(object sender, EventArgs e)
2: {
3:     polinom = new PolinomulHermite(0,
4:         new[] {(decimal)Math.PI / 6, (decimal)Math.PI / 2},
5:         new[] {1M / 2, 1}, new[] {(decimal)Math.Sqrt(3) / 2, 0});
6:     polinom.CalculeazaPuncte();
7:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.F[0]);
8: }
9: private void BernsteinELaX_Click(object sender, EventArgs e)
10: {
11:     polinom = new PolinomulHermite(10,
12:         new[] {0M, 1}, new[] {1M, (decimal)Math.E},
13:         x => (decimal)Math.Pow(Math.E, (double)x);
14:     polinom.CalculeazaPuncte();
15:     DeseneazaGrafic(new Pen(Color.Green, 3), polinom.u, polinom.F[1]);
16:     DeseneazaGrafic(new Pen(Color.Red, 3), polinom.u, polinom.F[3]);
17:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.F[6]);
18:     DeseneazaGrafic(new Pen(Color.Black, 3), polinom.u,polinom.F[10]);
19: }
```


Polinoame de Interpolare
1.00
f
P
0.50
0.52
1.57
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie
Polinomul Hermite:
Sinus
Radical


Polinoame de Interpolare
2.72
f
B1f
B3f
B6f
B10f
1.00
0.00
1.00
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie
Polinomul Hermite:
Sinus
Radical
Polinomul Bernstein:
e^x
1 + Radical

Capitolul 11

Laboratorul 11. Regresia liniară, pătratică şi funcţii spline liniare

METODA CELOR MAI MICI PĂTRATE

DREAPTA DE REGRESIE

Algoritmul dreptei de regresie
I. Date de intrare:
n numărul datelor
$x_i, \quad i = \overline{0,n}, \quad$ valorile mărimii x
$y_i, \quad i = \overline{0,n}, \quad$ valorile mărimii y
II. Date de ieșire:
a, b coeficienții din ecuația carteziană a dreptei de regresie
III. Pașii algoritmului:
Pasul 1. $S1 := 0$, $S2 := 0$, $T1 := 0$, $T2 := 0$;
Pentru $i = \overline{0,n}$ calculează $S1 := S1 + x_i$; $S2 := S2 + x_i \cdot x_i$;
$T1 := T1 + y_i$; $T2 := T2 + x_i \cdot y_i$;
Pasul 2. Calculează

$$d := (n+1) \cdot S2 - S1 \cdot S1$$

$$d_1 := (n+1) \cdot T2 - S1 \cdot T1$$

$$d_2 := S2 \cdot T1 - T2 \cdot S1$$

Pasul 3. Calculează

$$a := \frac{d_1}{d}, \quad b := \frac{d_2}{d}$$

Pasul 4. Tipărește a, b;
Pasul 5. Trasarea dreptei
Calculează $h = \frac{x_n - x_0}{1000}$
pentru $j = \overline{0,1000}$ calculează
$u[j] := x_0 + j \cdot h$

$$q[j] := a \cdot u[j] + b$$
Trasează punctul $(u[j], q[j])$;

Stop.

Exemplu numeric:
Fie tabelul de valori ale funcției $y = f(x)$ cu n=5.

x_i :	1	3	4	6	8	9
y_i :	1	2	4	4	5	3

Să se determine dreapta de regresie de ecuație $y = ax + b$ ce ajustează datele din tabel și să se traseze. (Se găsesc $a = \frac{95}{281}$ și $b = \frac{399}{281}$).

PARABOLA DE REGRESIE

Algoritmul parabolei de regresie
I. Date de intrare:
n numărul datelor
x_i, $i = \overline{0, n}$, valorile mărimii x
y_i, $i = \overline{0, n}$, valorile mărimii y
II. Date de ieșire:
a, b, c coeficienții din ecuația cartezianǎ a parabolei de regresie
III. Pașii algoritmului:
Pasul 1. $S1 := 0$, $S2 := 0$, $S3 := 0$, $S4 := 0$, $T1 := 0$, $T2 := 0$, $T3 := 0$;
Pentru $i = \overline{0, n}$ calculează $S1 := S1 + x_i$; $S2 := S2 + x_i \cdot x_i$;
$S3 := S3 + x_i \cdot x_i \cdot x_i$; $S4 := S4 + x_i \cdot x_i \cdot x_i \cdot x_i$;
$T1 := T1 + y_i$; $T2 := T2 + x_i \cdot y_i$; $T3 := T3 + x_i \cdot x_i \cdot y_i$
Calculează

$$d := (n+1) \cdot S2 \cdot S4 + 2 \cdot S1 \cdot S2 \cdot S3 - S2 \cdot S2 \cdot S2 - S1 \cdot S1 \cdot S4 - (n+1) \cdot S3 \cdot S3$$

$$d_1 := (n+1) \cdot S2 \cdot T3 + S1 \cdot S2 \cdot T2 + S1 \cdot S3 \cdot T1 - S2 \cdot S2 \cdot T1 - S1 \cdot S1 \cdot T2 - (n+1) \cdot S3 \cdot T2$$

$$d_2 := (n+1) \cdot T2 \cdot S4 + S4 + S2 \cdot S3 \cdot T1 + S1 \cdot S2 \cdot T3 - S2 \cdot S2 \cdot T2 - S1 \cdot S4 \cdot T1 - (n+1) \cdot S3 \cdot T3$$

$$d_3 := S2 \cdot S4 \cdot T1 + S1 \cdot S3 \cdot T3 + S2 \cdot S3 \cdot T2 - S2 \cdot S2 \cdot T3 - S1 \cdot S4 \cdot T2 - S3 \cdot S3 \cdot T1$$

Pasul 2. Calculează
$$a := \frac{d_1}{d}, \quad b := \frac{d_2}{d}, \quad c := \frac{d_3}{d}$$
Pasul 3. Tipărește a, b, c;
Pasul 4. Trasarea parabolei
Calculează $h = \frac{x_n - x_0}{1000}$
pentru $j = \overline{0, 1000}$ calculează
$$u[j] := x_0 + j \cdot h$$
$$q[j] := a \cdot u[j] \cdot u[j] + b \cdot u[j] + c$$
Trasează punctul $(u[j], q[j])$;

Stop.

Exemplu numeric:
Fie tabelul de valori ale funcției $y = f(x)$ cu n=6.

x_i :	-3	-2	-1	0	1	2	3
y_i :	6	4	1	2	3	8	11

Să se determine parabola de regresie de ecuație $y = ax^2 + bx + c$ ce ajustează datele din tabel si să se traseze. (Se găsesc $a = \frac{64}{84}$, $b = \frac{25}{28}$ și $c = \frac{40}{21}$).

FUNCȚII SPLINE POLIGONALE

Algoritmul funcției spline poligonale
I. Date de intrare: a, b capetele intervalului
n numărul de subintervale ale diviziunii
$x[i]$, $i = \overline{0,n}$ nodurile de interpolare
$y[i]$, $i = \overline{0,n}$ valorile pe noduri
II. Date de ieșire: punctele $(u[k], s(k))$, $k = \overline{0,1000}$ în care se calculează funcția spline
III. Pașii algoritmului
1. Calculează $h =: \frac{b-a}{1000}$. Pentru $i = \overline{1,n}$ calculează $h[i] = x[i] - x[i-1]$
2. Pentru $k = \overline{0,1000}$ calculează $u[k] := a + k \cdot h$
Pentru $j = \overline{1,n}$
Dacă $x[j-1] \leq u[k] \leq x[j]$
atunci calculează

$$s[k] := y[j-1] + \frac{y[j] - y[j-1]}{h[j]} \cdot (u[k] - x[j-1])$$

4. Pentru $k = \overline{0,1000}$ tipărește (desenează) punctele $(u[k], s[k])$; STOP.
Exemplu numeric:
La momentele 7.5 (adică ora 7^{30}), 10.5 (ora 10^{30}), 13, 15.5 (ora 15^{30}), 18, 21, 24 și 27 (adică ora 3 A. M. a doua zi) s-au măsurat valorile glicemiei, obținându-se 130, 121, 128, 96, 122, 138, 114, 90 (măsurate în mg/dl). Să se aproximeze glicemia acestui pacient de la orele 12, 14, și 23 folosind funcția spline poligonală de interpolare.
Datele de intrare sunt: $n = 7$, $a = 7.5$, $b = 27$, valorile sunt date în tabel

x_i, $i = \overline{0,n}$:	7.5	10.5	13	15.5	18	21	24	27
y_i, $i = \overline{0,n}$:	130	121	128	96	122	138	114	90

FUNCȚII SPLINE PĂTRATICE

Algoritmul funcției spline pătratice
I. Date de intrare: a, b capetele intervalului
n numărul de subintervale ale diviziunii
$x[i]$, $i = \overline{0,n}$ niodurile de interpolare
$y[i]$, $i = \overline{0,n}$ valorile pe noduri
II. Date de ieșire: punctele $(u[k], s(k))$, $k = \overline{0,1000}$ în care se calculează funcția spline

III. Pașii algoritmului

1. Calculează $h =: \frac{b-a}{1000}$. Pentru $i = \overline{1,n}$ calculează $h[i] = x[i] - x[i-1]$

2. Calculează

$$m[0] := \frac{2 \cdot h[1] + h[2]}{h[1] \cdot (h[1] + h[2])} \cdot (y[1] - y[0]) - \frac{h[1]}{h[1] \cdot (h[1] + h[2])} \cdot (y[2] - y[1])$$

Pentru $i = \overline{1,n}$ calculează

$$m[i] = \frac{2}{h[i]} \cdot (y[i] - y[i-1]) - m[i-1]$$

3. Pentru $k = \overline{0,1000}$ calculează $u[k] := a + k \cdot h$

Pentru $j = \overline{1,n}$

Dacă $x[j-1] \leq u[k] \leq x[j]$

atunci calculează

$$s[k] := \frac{m[j] - m[j-1]}{2h[j]} \cdot (u[k] - x[j-1])^2 + m[j-1] \cdot (u[k] - x[j-1]) + y[j-1]$$

4. Pentru $k = \overline{0,1000}$ tipăreste (desenează) punctele $(u[k], s[k])$; STOP.

Exemplu numeric:

La momentele 7.5 (adică ora 7^{30}), 10.5 (ora 10^{30}), 13, 15.5 (ora 15^{30}), 18, 21, 24 și 27 (adică ora 3 A. M. a doua zi) s-au măsurat valorile glicemiei, obținându-se 130, 121, 128, 96, 122, 138, 114, 90 (măsurate in mg/dl). Să se aproximeze glicemia acestui pacient de la orele 12, 14, și 23 folosind funcția spline pătratică de interpolare.

Datele de intrare sunt: $n = 7$, $a = 7.5$, $b = 27$, valorile sunt date în tabel

x_i, $i = \overline{0,n}$:	7.5	10.5	13	15.5	18	21	24	27
y_i, $i = \overline{0,n}$:	130	121	128	96	122	138	114	90

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru fiecare dintre cele 4 procedee prezentate, clasele arată foarte simplu și similar, doar algoritmul este diferit, așa că vom arăta doar clasa pentru unul dintre procedee.

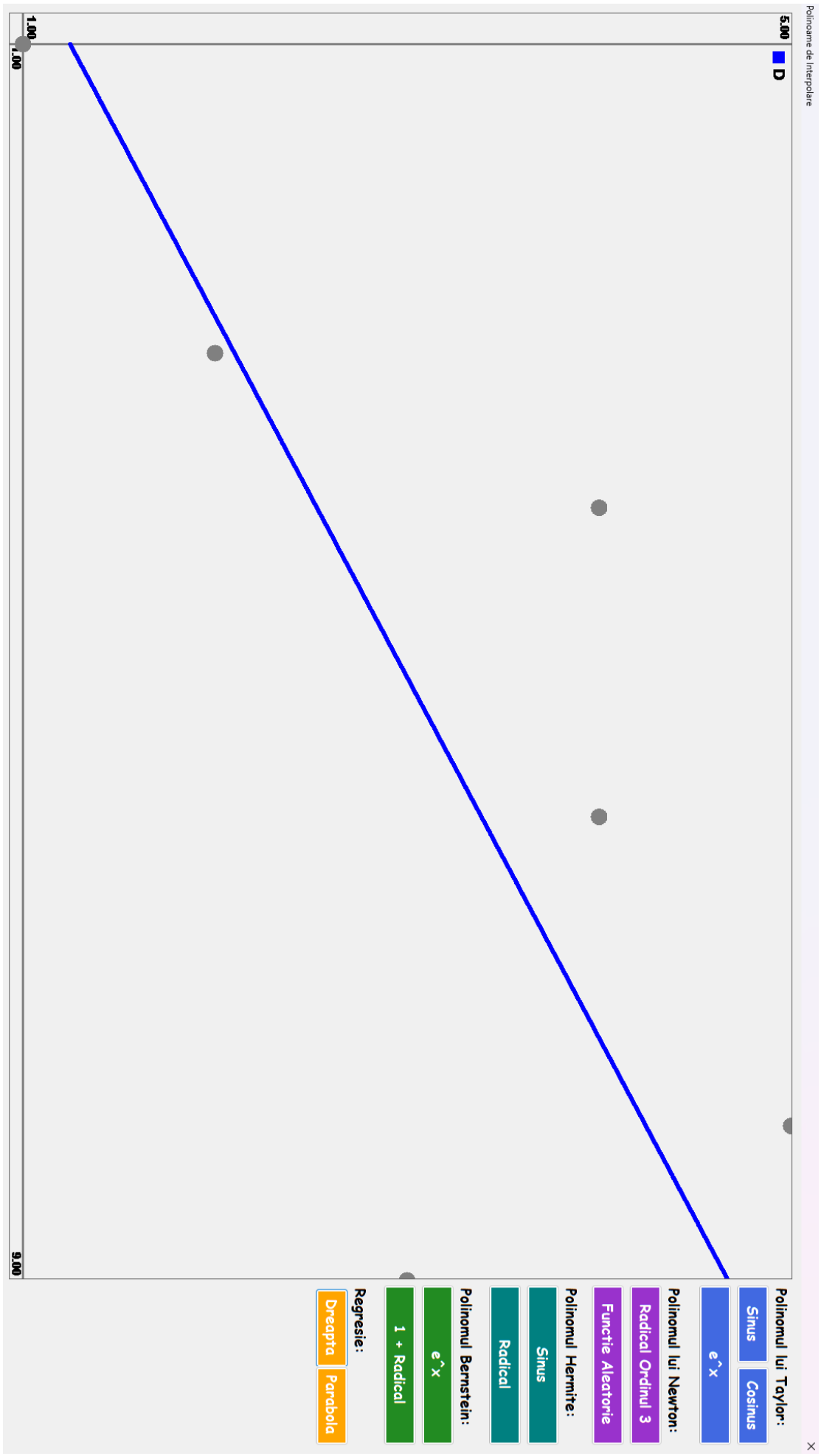
Algorithm 32 Clasa ParabolaRegresie

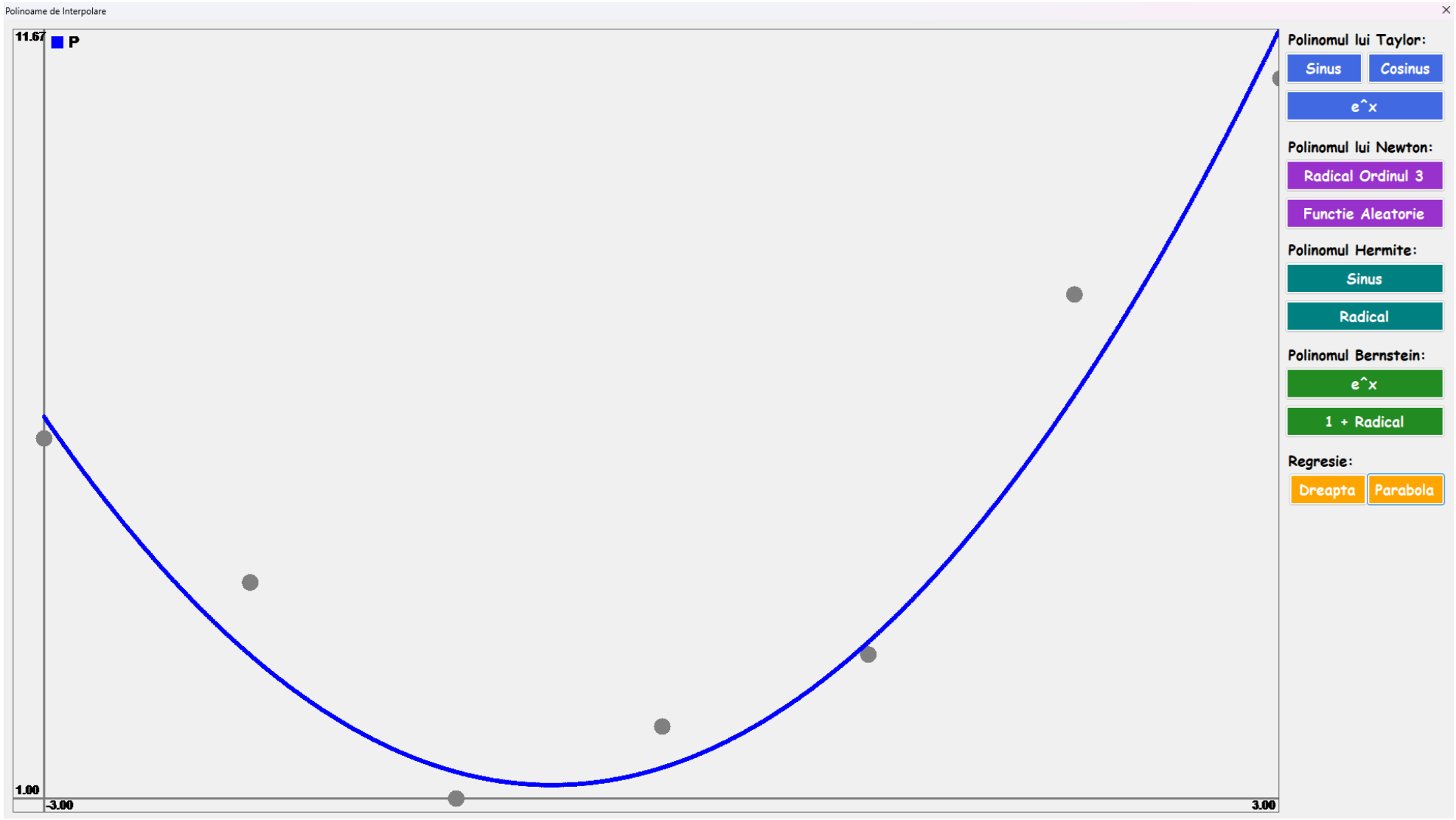
```
1: public class ParabolaRegresie :  Polinom
2: {
3:     public ParabolaRegresie(int n, decimal[] x, decimal[] y)
4:         :  base(n, x, y) { }
5:     public override void CalculeazaPuncte()
6:     {
7:         // Algoritm
8:     }
9: }
```

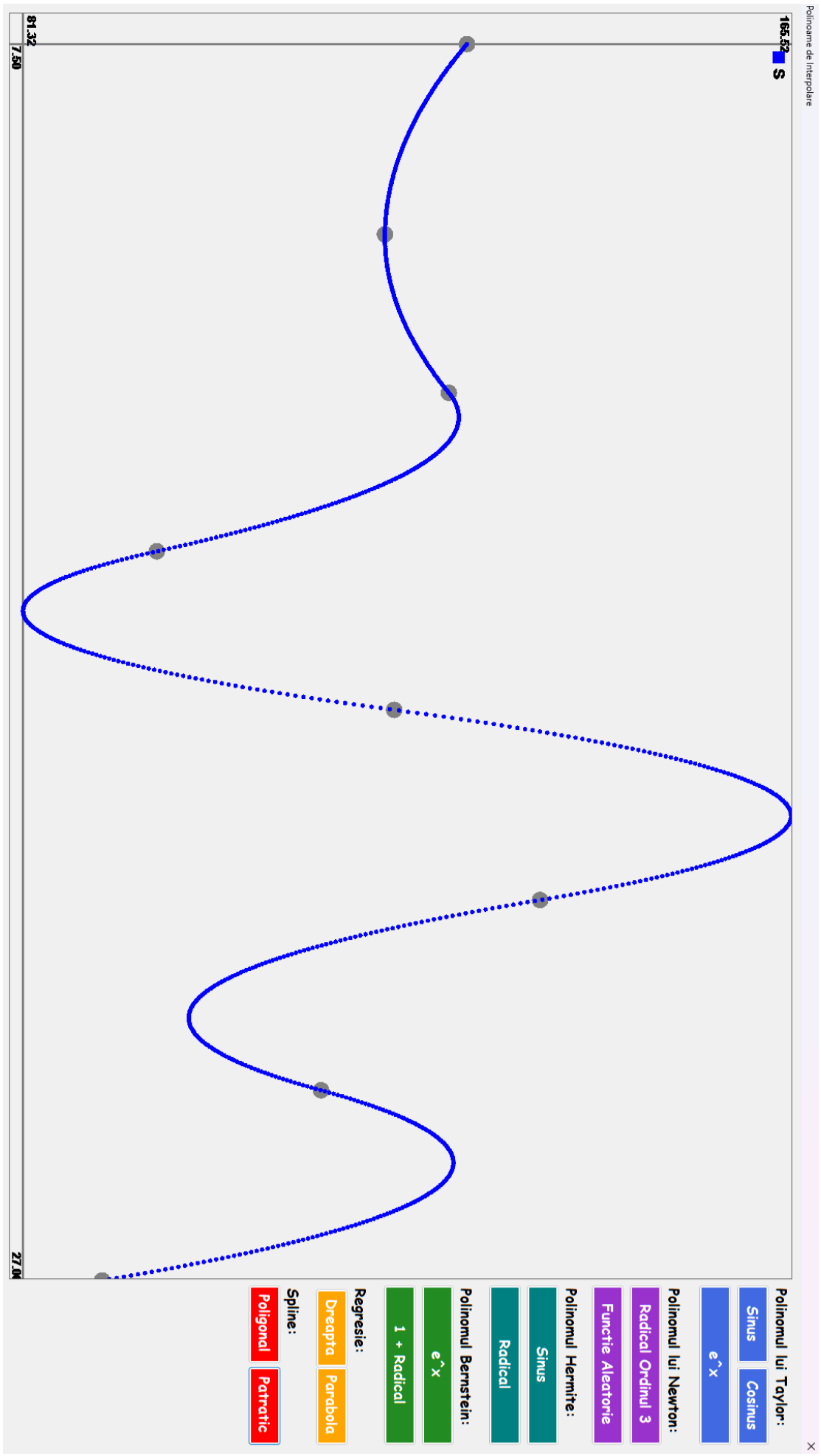
În implementarea exemplelor, avem doar un grafic de afișat, și nu a mai fost nevoie să salvăm valorile în F, deci putem folosi direct f.

Algorithm 33 Implementarea metodei click a butoanelor pentru afișarea exemplelor

```
1: private void DreaptaRegresie_Click(object sender, EventArgs e)
2: {
3:     polinom = new DreaptaRegresie(6, new[] {1M, 3, 4, 6, 8, 9},
4:                                      new[] {1M, 2, 4, 4, 5, 3});
5:     polinom.CalculeazaPuncte();
6:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.f);
7: }
8: private void ParabolaRegresie_Click(object sender, EventArgs e)
9: {
10:     polinom = new ParabolaRegresie(7, new[] {-3M, -2, -1, 0, 1, 2, 3},
11:                                         new[] {6M, 4, 1, 2, 3, 8, 11});
12:     polinom.CalculeazaPuncte();
13:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.f);
14: }
15: private void SplinePatratic_Click(object sender, EventArgs e)
16: {
17: polinom = new SplinePatratic(7, new[]{7.5, 10.5, 13, 15.5, 18, 21, 24, 27},
18:                               new[] {130M, 121, 128, 96, 122, 138, 114, 90});
19:     polinom.CalculeazaPuncte();
20:     DeseneazaGrafic(new Pen(Color.Blue, 3), polinom.u, polinom.f);
21: }
```


Polinoame de Interpolare
5.00
D
1.00
1.00
9.00
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie
Polinomul Hermite:
Sinus
Radical
Polinomul Bernstein:
e^x
1 + Radical
Regresie:
Dreapta
Parabola


Polinoame de Interpolare
11.67
P
1.00
-3.00
3.00
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie
Polinomul Hermite:
Sinus
Radical
Polinomul Bernstein:
e^x
1 + Radical
Regresie:
Dreapta
Parabola


Polinoame de Interpolare
165.52
S
81.32
7.50
27.04
Polinomul lui Taylor:
Sinus
Cosinus
e^x
Polinomul lui Newton:
Radical Ordinul 3
Functie Aleatorie
Polinomul Hermite:
Sinus
Radical
Polinomul Bernstein:
e^x
1 + Radical
Regresie:
Dreapta
Parabola
Spline:
Poligonal
Patratic

Capitolul 12

Laboratorul 12. Funcţii spline cubice

FUNCȚII SPLINE CUBICE

Algoritmul funcției spline cubice naturale generată de condiții bilocale
I. Date de intrare: a, b capetele intervalului
n numărul de subintervale ale diviziunii
$x[i]$, $i = \overline{0,n}$ nodurile de interpolare
$y[i]$, $i = \overline{0,n}$ valorile pe noduri
II. Date de ieșire: punctele $(u[k], s[k])$, $k = \overline{0,1000}$ în care se calculează funcția spline
III. Pașii algoritmului
1. Calculează $q =: \frac{b-a}{1000}$. Pentru $i = \overline{1,n}$ calculează $h_i = x[i] - x[i-1]$
Pentru $i = \overline{1,n-1}$ calculează

$$a_i := 2$$

și

$$d_i := \frac{6}{h_i + h_{i+1}} \cdot \left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i} \right).$$

Pentru $i = \overline{2,n-2}$ calculează

$$b_i := \frac{h_i}{h_i + h_{i+1}}, \quad c_i := 1 - b_i$$

Execută

$$b_{n-1} := \frac{h_{n-1}}{h_{n-1} + h_n}, \quad c_1 := \frac{h_2}{h_1 + h_2}$$

2. Calculează, $\alpha_1 := \frac{c_1}{a_1}$ și pentru $i = \overline{2,n-2}$, calculează

$$\omega_i =: a_i - \alpha_{i-1} \cdot b_i, \qquad \alpha_i := \frac{c_i}{\omega_i},$$

3. Calculează

$$\omega_{n-1} := a_{n-1} - \alpha_{n-2} \cdot b_{n-1}.$$

4. Calculează $z_1 := \frac{d_1}{2}$ și pentru $i = \overline{2, n-1}$, calculează

$$z_i := \frac{d_i - b_i \cdot z_{i-1}}{\omega_i}.$$

5. Calculează $M_{n-1} := z_{n-1}$ și pentru $i = \overline{n-2, 1}$, calculează prin recurență înapoi,

$$M_i := z_i - \alpha_i \cdot M_{i+1}.$$

Calculează $M_0 = M_n = 0$.

6. Pentru $k = \overline{0, 1000}$ calculează $u[k] := a + k \cdot q$

Pentru $j = \overline{1, n}$

Dacă $x[j-1] \leq u[k] \leq x[j]$

atunci calculează

$$s[k] := \left[\frac{(u[k] - x_{j-1})^3}{6h_j} - \frac{h_j\,(u[k] - x_{j-1})}{6}\right] \cdot M_j + \left[\frac{(x_j - u[k])^3}{6h_j} - \frac{h_i\,(x_j - u[k])}{6}\right] \cdot M_{j-1} +$$

$$+ \frac{x_j - u[k]}{h_j} \cdot y_{j-1} + \frac{u[k] - x_{j-1}}{h_j} \cdot y_j$$

7. Pentru $k = \overline{0, 1000}$ tipărește (desenează) punctele $(u[k], s[k])$; STOP.

Exemplu numeric:

La momentele 7.5 (adică ora 7^{30}), 10.5 (ora 10^{30}), 13, 15.5 (ora 15^{30}), 18, 21, 24 și 27 (adică ora 3 A. M. a doua zi) s-au măsurat valorile glicemiei, obținându-se 130, 121, 128, 96, 122, 138, 114, 90 (măsurate în mg/dl). Să se aproximeze glicemia acestui pacient de la orele 12, 14, și 23 folosind funcția spline cubică naturală de interpolare.

Datele de intrare sunt: $n = 7$, $a = 7.5$, $b = 27$, valorile sunt date în tabel

x_i, $i = \overline{0,n}$: 7.5	10.5	13	15.5	18	21	24	27
y_i, $i = \overline{0,n}$: 130	121	128	96	122	138	114	90

Algoritmul metodei de interpolare a lui Akima

I. Date de intrare: a, b capetele intervalului

n numărul de subintervale ale diviziunii

$x[i]$, $i = \overline{0, n}$ niodurile de interpolare

$y[i]$, $i = \overline{0, n}$ valorile pe noduri

II. Date de ieșire: punctele $(u[k], s(k))$, $k = \overline{0, 1000}$ în care se calculează funcția spline

III. Pașii algoritmului

1. Calculează $h =: \frac{b-a}{1000}$. Pentru $i = \overline{0, n-1}$ calculează pantele

$$p_i := \frac{y_{i+1} - y_i}{x_{i+1} - x_i}$$

2. Calculează

$$u := 3p_0 - 2p_1$$

$$v := 2p_0 - p_1$$

$$p_n := 2p_{n-1} - p_{n-2}$$

$$p_{n+1} := 3p_{n-1} - 2p_{n-2}$$

3. Dacă $|p_1 - p_0| + |v - u| \neq 0$ atunci calculează

$$m_0 := \frac{|p_1 - p_0| \cdot v + |v - u| \cdot p_0}{|p_1 - p_0| + |v - u|}$$

altfel

$$m_0 := \frac{v + p_0}{2}$$

și dacă $|p_2 - p_1| + |p_0 - v| \neq 0$ atunci calculează

$$m_1 := \frac{|p_2 - p_1| \cdot p_0 + |p_0 - v| \cdot p_1}{|p_2 - p_1| + |p_0 - v|}$$

altfel

$$m_1 := \frac{p_0 + p_1}{2}$$

4. Pentru $i = \overline{2, n}$, dacă $|p_{i+1} - p_i| + |p_{i-1} - p_{i-2}| \neq 0$ atunci calculează

$$m_i := \frac{|p_{i+1} - p_i| \cdot p_{i-1} + |p_{i-1} - p_{i-2}| \cdot p_i}{|p_{i+1} - p_i| + |p_{i-1} - p_{i-2}|}$$

altfel,

$$m_i := \frac{p_{i-1} + p_i}{2}$$

5. Pentru $k = \overline{0, 1000}$ calculează $u[k] := a + k \cdot h$
Pentru $j = \overline{1, n}$
Dacă $x[j-1] \leq u[k] \leq x[j]$
atunci calculează

$$s[k] := \frac{(x_j - u[k])^2 (u[k] - x_{j-1})}{(x_j - x_{j-1})^2} \cdot m_{j-1} - \frac{(u[k] - x_{j-1})^2 (x_j - u[k])}{(x_j - x_{j-1})^2} \cdot m_j +$$

$$+ \frac{(x_j - u[k])^2 [2(u[k] - x_{j-1}) + (x_j - x_{j-1})]}{(x_j - x_{j-1})^3} \cdot y_{j-1} + \frac{(u[k] - x_{j-1})^2 [2(x_j - u[k]) + (x_j - x_{j-1})]}{(x_j - x_{j-1})^3} \cdot y_j$$

6. Pentru $k = \overline{0, 1000}$ tipărește (desenează) punctele $(u[k], s[k])$; STOP.

Exemplu numeric:

La momentele 7.5 (adică ora 7^{30}), 10.5 (ora 10^{30}), 13, 15.5 (ora 15^{30}), 18, 21, 24 și 27 (adică ora 3 A. M. a doua zi) s-au măsurat valorile glicemiei, obținându-se 130, 121, 128, 96, 122, 138, 114, 90 (măsurate în mg/dl). Să se aproximeze glicemia acestui pacient de la orele 12, 14, și 23 folosind funcția spline Akima cubică de interpolare.

Datele de intrare sunt: $n = 7$, $a = 7.5$, $b = 27$, valorile sunt date în tabel

x_i, $i = \overline{0, n}$: 7.5	10.5	13	15.5	18	21	24	27
y_i, $i = \overline{0, n}$: 130	121	128	96	122	138	114	90

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Codul este din nou foarte similar cu celelalte spline implementate, așa că de data aceasta venim cu o provocare: afișați toate spline-urile deodată pentru a le putea compara, așa cum puteți vedea în imagine.



Capitolul 13

Laboratorul 13. Cuadraturi de tip trapezoidal

FORMULE DE CUADRATURĂ DE TIP TRAPEZOIDAL

1. **Algoritmul cuadraturii dreptunghiului:**
I. Date de intrare:
a, b: capetele intervalului
n: numărul de subintervale al diviziunii
f : expresia funcției de sub integrală
II. Date de ieșire: $D-$valoarea aproximativă a acestei integrale.
III. Pașii algoritmului:
1. Se citesc datele: a, b, n
2. Se crează procedura de introducere a funcției f
3. Se construiesc nodurile diviziunii:
Pentru $i = \overline{0, n}$ calculează

$$x[i] = a + \frac{i \cdot (b-a)}{n}$$

4. Calculul sumei integrale:
$S := 0$
Pentru $i = \overline{1, n}$ calculează

$$S := S + f\left(x[i]\right)$$

5. Calculul aproximației integralei:

$$D := \left(\frac{(b-a)}{n}\right) \cdot S$$

6. Afișarea rezultatului: Tipărește D; STOP.

Exemple numerice:

1. Să se utilizeze formula de cuadratură a dreptunghiului cu $a = 0$, $b = 1$, $f(x) = \frac{1}{x+1}$, pentru n=100 la calculul integralei

$$\int_0^1 \frac{1}{x+1} dx$$

și să se compare rezultatul cu valoarea exactă a acestei integrale, $\ln 2 \simeq 0.69315$.

2. Să se utilizeze formula de cuadratură a dreptunghiului $a = 0$, $b = 1$, $f(x) = \frac{4}{x^2+1}$, pentru n=100 la calculul integralei

$$\pi = \int_0^1 \frac{4}{x^2+1} dx$$

și să se compare rezultatul cu valoarea exactă a acestei integrale $\pi \simeq 3.14159265359$.

Algoritmul cuadraturii trapezului

I. Date de intrare:

a, b: capetele intervalului

n: numărul de subintervale al diviziunii

f : expresia funcției de sub integrală

II. Date de ieșire: $T-$valoarea aproximativă a acestei integrale.

III. Pașii algoritmului:

1. Se citesc datele: a,b,n
2. Se crează procedura de introducere a funcției f
3. Se construiesc nodurile diviziunii:

Pentru $i = \overline{0, n}$ calculează

$$x[i] = a + \frac{i \cdot (b-a)}{n}$$

4. Calculul sumei integrale:

$S := 0$

Pentru $i = \overline{1, n}$ calculează

$$S := S + (f(x[i]) + f(x[i-1]))$$

5. Calculul aproximației integralei:

$$T := \left(\frac{(b-a)}{2 \cdot n}\right) \cdot S$$

6. Afișarea rezultatului: Tipăreste T; STOP.

Exemple numerice:

1. Să se utilizeze formula de cuadratură a trapezului cu $a = 0$, $b = 1$, $f(x) = \frac{1}{x+1}$, pentru n=10 și n=100 la calculul integralei

$$\int_0^1 \frac{1}{x+1} dx$$

și să se compare rezultatul cu valoarea exacta a acestei integrale, $\ln 2 \simeq 0.69315$. Să se compare rezultatul cu cel furnizat de cuadratura dreptunghiului.

2. Să se utilizeze formula de cuadratură a trapezului cu $a = 0$, $b = 1$, $f(x) = \frac{4}{x^2+1}$, pentru n=10 și n=100 la calculul integralei

$$\int_0^1 \frac{4}{x^2+1} dx$$

și să se compare rezultatul cu valoarea exactă a acestei integrale $\pi \simeq 3.14159265359$. Să se compare rezultatul cu cel furnizat de cuadratura dreptunghiului.

3. Să se utilizeze formula de cuadratura a trapezului cu $a = 0$, $b = 1.5$, $f(x) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{x^2}{2}}$ pentru n=10 și n=100, la calculul integralei lui Laplace

$$\frac{1}{\sqrt{2\pi}} \int_0^{1.5} e^{-\frac{x^2}{2}} dx.$$

4. Să se utilizeze formula de cuadratură a trapezului pentru n=20 la calculul integralei

$$\int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x} dx$$

cu $\epsilon = 0,016729$ și apoi să se calculeze o aproximație pentru lungimea orbitei Pământului în jurul Soarelui

$$L(P) = 4c \int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x} dx,$$

unde $c = 149,6 \times 10^6$ km. Se va lua $a = 0$, $b = \frac{\pi}{2}$, $f(x) = 4c \cdot \sqrt{1 - \epsilon^2 \sin^2 x}$

Algoritmul cuadraturii corectate a trapezului

I. Date de intrare:

a,b capetele intervalului de integrare

dfa, dfb : valorile derivatei pe noduri

n, numărul nodurilor

f funcția ce se integrează

II. Date de ieșire : S, valoarea aproximativă a integralei $\int_a^b f(x)dx$

III. Pașii algoritmului

1. Pentru $i = \overline{0, n}$ calculează

$$x_i := a + i \cdot \frac{b-a}{n}$$

2. Calculează

$$T := 0$$

Pentru $i = \overline{1,n}$ calculează

$$T := T + [f(x_{i-1}) + f(x_i)]$$

3. Calculează

$$S := \left(\frac{(b-a)}{2 \cdot n}\right) \cdot T - \left(\frac{(b-a)^2}{12 \cdot n^2}\right) \cdot (dfb - dfa)$$

4. Tipărește S. STOP.

Exemple numerice:

1. Să se utilizeze formula de cuadratură corectată a trapezului cu $a = 0$, $b = 1$, $f(x) = \frac{1}{x+1}$, pentru n=10 și $dfa = -1$, $dfb = -\frac{1}{4}$, la calculul integralei

$$\int_0^1 \frac{1}{x+1} dx$$

și să se compare rezultatul cu valoarea exactă a acestei integrale, $\ln 2 \simeq 0.69315$. Să se compare rezultatul cu cel furnizat de cuadratura trapezului.

2. Să se utilizeze formula de cuadratură corectată a trapezului cu $a = 0$, $b = 1$, $f(x) = \frac{4}{x^2+1}$, pentru n=10, n=100, și $dfa = 0$, $dfb = -2$, la calculul integralei

$$\int_0^1 \frac{4}{x^2+1} dx$$

și să se compare rezultatul cu valoarea exactă a acestei integrale $\pi \simeq 3.14159265359$. Să se compare rezultatul cu cel furnizat de cuadratura trapezului.

3. Să se utilizeze formula de cuadratură corectată a trapezului cu $a = 0$, $b = 1.5$, $f(x) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{x^2}{2}}$ pentru n=10, n=100 și $dfa = 0$, $dfb = -0.19428$, la calculul integralei lui Laplace

$$\frac{1}{\sqrt{2\pi}} \int_0^{1.5} e^{-\frac{x^2}{2}} dx.$$

Să se compare rezultatul cu cel furnizat de cuadratura trapezului.

4. Să se utilizeze formula de cuadratură corectată a trapezului pentru n=20 și $dfa = 0$, $dfb = 0$ la calculul integralei

$$\int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x} dx$$

cu $\epsilon = 0,016729$ și apoi să se calculeze o aproximație pentru lungimea orbitei Pământului în jurul Soarelui

$$L(P) = 4c \int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x} dx,$$

unde $c = 149,6 \times 10^6$ km. Să se compare rezultatul cu cel furnizat de cuadratura trapezului. Se va lua $a = 0$, $b = \frac{\pi}{2}$, $f(x) = 4c \cdot \sqrt{1 - \epsilon^2 \sin^2 x}$

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Pentru implementarea metodelor de aproximare a integralelor definite, se utilizează o clasă de bază denumită FormulaDeCuadratura, care conține elementele comune tuturor formulelor de cuadratură. Aceasta gestionează intervalul de integrare, funcția ce se integrează, numărul subdiviziunilor, valorile nodurilor subdiviziunilor și calculul integralei. Fiecare metodă particulară de cuadratură va moșteni această clasă și va implementa propriul algoritm de aproximare.

Constructorul clasei FormulaDeCuadratura inițializează toate datele comune necesare aproximării integralei: intervalul de integrare $[a, b]$, numărul de subdiviziuni n, funcția ce se integrează. În plus, constructorul construiește vectorul x, care conține nodurile discretizării intervalului. Aceste noduri sunt distribuite uniform și sunt utilizate ulterior de formulele de cuadratură pentru evaluarea funcției și calculul aproximării integralei.

Pentru calcularea integralei efective, fiecare clasă derivată va suprascrie metoda AproximareIntegrala(), care este o metodă abstractă definită în această clasă.

Algorithm 34 Clasa FormulaDeCuadratura

```
public abstract class FormulaDeCuadratura
{
    protected decimal a, b;
    protected int n;
    protected decimal[] x;
    protected decimal Integrala;
    protected Func<decimal, decimal> f;
    public FormulaDeCuadratura(decimal a, decimal b, int n,
        Func<decimal, decimal> f)
    {
        this.a = a;
        this.b = b;
        this.n = n;
        this.f = f;
        x = new decimal[n + 1];
        for (int i = 0; i <= n; i++)
        {
            x[i] = a + i * (b - a) / n;
        }
    }
    protected override void AproximareIntegrala()
}
```

În cazul calculului integralei unei funcții prin formulei dreptunghiului, clasa FormulaDreptunghiului moștenește clasa FormulaDeCuadratura și implementează algoritmul formulei dreptunghiului pentru aproximarea unei integrale definite.

Constructorul clasei FormulaDreptunghiului primește intervalul de integrare, numărul de diviziuni și funcția care ajută la integrare. După inițializarea acestor date, constructorul apelează metoda AproximareIntegrala().

Algorithm 35 Clasa FormulaDreptunghiului

```
1: public class FormulaDreptunghiului :  FormulaDeCuadratura
2: {
3:     public FormulaDreptunghiului(decimal a, decimal b, int n,
4:         Func<decimal, decimal> f) :  base(a, b, n, f)
5:     {
6:         AproximareIntegrala();
7:     }
8:     protected override void AproximareIntegrala()
9:     {
10:        //Algoritm
11:    }
12: }
```

În cazul calculului integralei unei funcții prin formula trapezului, se construiește clasa FormulaTrapezului ce moștenește clasa FormulaDeCuadratura și implementează formula trapezului pentru aproximarea integralelor definite.

Constructorul primește intervalul de integrare, numărul de diviziuni, funcția și valoarea exactă utilizată pentru comparație, după care inițializează datele și execută calculele necesare.

Algorithm 36 Clasa FormulaTrapezului

```
1: public class FormulaTrapezului :  FormulaDeCuadratura
2: {
3:     public FormulaTrapezului(decimal a, decimal b, int n,
4:         Func<decimal, decimal> f) :  base(a, b, n, f)
5:     {
6:         AproximareIntegrala();
7:     }
8:     protected override void AproximareIntegrala()
9:     {
10:        // Algoritm
11:    }
12: }
```

În cazul formulei corectate a trapezului, se construiește Clasa FormulaCorectataA-Trapezului ce moștenește clasa FormulaTrapezului și implementează formula corectată a trapezului pentru aproximarea integralelor definite.

Constructorul clasei FormulaCorectataATrapezului primește intervalul de integrare, numărul de diviziuni, funcția și valorile derivatei funcției în capetele intervalului. După inițializarea acestor date sunt executate calculele necesare aproximării integralei.

Metoda AproximareIntegrala() reutilizează mai întâi algoritmul formulei trapezului prin apelul base.AproximareIntegrala(). Ulterior, valoarea obținută este corectată prin termenul specific formulei corectate a trapezului, în care intervin valorile derivatei funcției în capetele intervalului.

Algorithm 37 Clasa FormulaCorectataATrapezului

```
1: public class FormulaCorectataATrapezului :  FormulaTrapezului
2: {
3:     protected decimal dfa;
4:     protected decimal dfb;
5:     public FormulaCorectataATrapezului(decimal a, decimal b, int n,
6:         Func<decimal, decimal> f, decimal dfa, decimal dfb)
7:         :  base(a, b, n, f)
8:     {
9:         this.dfa = dfa;
10:        this.dfb = dfb;
11:        AproximareIntegrala();
12:    }
13:    protected override void AproximareIntegrala()
14:    {
15:        base.AproximareIntegrala();
16:        Integrala = Integrala - (b - a) * (b - a) / (12 * n * n)
17:                                * (dfb - dfa);
18:    }
19: }
```

Capitolul 14

Laboratorul 14. Cuadraturi de tip Simpson şi Newton

FORMULE DE CUADRATURĂ DE TIP SIMPSON ȘI NEWTON-COTES

Algoritmul formulei de cuadratură a lui Simpson
I. Date de intrare:
a,b capetele intervalului de integrare
n, numărul nodurilor
f funcția ce se integrează

II. Date de ieșire : S, valoarea aproximativă a integralei $\int_a^b f(x)dx$

III. Pașii algoritmului
1. Se citesc datele: a,b,n
2. Se crează procedura de introducere a funcției f
3. Se construiesc nodurile diviziunii:
Pentru $i = \overline{0,n}$ calculează

$$x[i] = a + \frac{i \cdot (b-a)}{n}$$

4. Calculul sumei integrale:
$T := 0$, $M := 0$
Pentru $i = \overline{1,n}$ calculează

$$T := T + (f(x[i]) + f(x[i-1]))$$

$$M := M + f\left(\frac{x[i-1] + x[i]}{2}\right)$$

5. Calculul aproximației integralei:

$$S := \left(\frac{(b-a)}{6 \cdot n}\right) \cdot (T + 4 \cdot M)$$

6. Afișarea rezultatului: Tipărește S; STOP.

Exemple numerice:

1. Se consideră funcția $f(x) = \frac{1}{1+x}$. Să se calculeze valoarea aproximativă a integralei $\int_0^1 f(x)\,dx$, cu n=4 și n=16, folosind formula generalizată a lui Simpson cu $a = 0$, $b = 1$, și $f(x) = \frac{1}{1+x}$.

2. Să se utilizeze formula de cuadratură clasică a lui Simpson pentru n=20 la calculul integralei

$$\int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x}\,dx$$

cu $\epsilon = 0,016729$ și apoi să se calculeze o aproximație pentru lungimea orbitei Pământului în jurul Soarelui

$$L(P) = 4c \int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x}\,dx,$$

unde $c = 149,6 \times 10^6$ km. Se consideră datele de intrare $a = 0$, $b = \frac{\pi}{2}$,

$$f(x) = 4c \cdot \sqrt{1 - \epsilon^2 \sin^2 x}.$$

Să se compare rezultatele obținute folosind cuadratura trapezului, cuadratura perturbată a trapezului și cuadratura lui Simpson.

Algoritmul formulei corectate de cuadratură a lui Simpson

I. Date de intrare:

a,b capetele intervalului de integrare

dfa, dfb : valorile derivatei pe capetele intervalului

n, numărul nodurilor

f funcția care se integrează

II. Date de ieșire : S, valoarea aproximativă a integralei $\int_a^b f(x)dx$

III. Pașii algoritmului

1. Se citesc datele: a, b, dfa, dfb, n
2. Se crează procedura de introducere a funcției f
3. Se construiesc nodurile diviziunii:

   Pentru $i = \overline{0, n}$ calculează

$$x[i] = a + \frac{i \cdot (b - a)}{n}$$

4. Calculul sumei integrale:

   $T := 0$, $M := 0$

Pentru $i = \overline{1,n}$ calculează

$$T := T + (f(x[i]) + f(x[i-1]))$$

$$M := M + f\left(\frac{x[i-1] + x[i]}{2}\right)$$

5. Calculul aproximației integralei:

$$S := \left(\frac{(b-a)}{30 \cdot n}\right) \cdot (7 \cdot T + 16 \cdot M) - \left(\frac{(b-a)^2}{60 \cdot n^2}\right) \cdot (dfb - dfa)$$

6. Afișarea rezultatului: Tipărește S; STOP.

Exemple numerice:

1. Se consideră funcția $f(x) = \frac{1}{1+x}$. Să se calculeze valoarea aproximativă a integralei $\int_0^1 f(x)\,dx$, cu $a = 0$, $b = 1$, $n = 10$ si $dfa = -1$, $dfb = -\frac{1}{4}$ folosind formula corectată de cuadratură a lui Simpson.

2. Să se utilizeze formula corectată de cuadratură a lui Simpson pentru n=10 și $a = 0$, $b = 1$, $f(x) = \frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{x^2}{2}}$, $dfa = 0$, $dfb = -\frac{1}{\sqrt{2\pi}} \cdot e^{-\frac{1}{2}} = -0.24197$, la calculul integralei lui Laplace

$$\frac{1}{\sqrt{2\pi}} \int_0^1 e^{-\frac{x^2}{2}} dx.$$

Să se compare rezultatul cu cel furnizat de formula de cuadratura a lui Simpson.

Algoritmul formulei de cuadratura a lui Newton:

I. Date de intrare:

a,b capetele intervalului de integrare

n, numărul nodurilor

f funcția ce se integrează

II. Date de ieșire : S, valoarea aproximativă a integralei $\int_a^b f(x)dx$

III. Pașii algoritmului

1. Se citesc datele: a,b,n
2. Se crează procedura de introducere a funcției f
3. Se construiesc nodurile diviziunii:

   Pentru $i = \overline{0,n}$ calculează

$$x[i] = a + \frac{i \cdot (b-a)}{n}$$

4. Se construiesc nodurile din interiorul subintervalelor

   Pentru $i = \overline{1,n}$ calculează

$$y[i] := x[i-1] + \frac{1}{3} \cdot (x[i] - x[i-1])$$

$$z[i] := x[i-1] + \frac{2}{3} \cdot (x[i] - x[i-1])$$

5. Calculul sumelor integrale
   Fie $T := 0,\ U := 0,\quad V := 0$
   Pentru $i = \overline{1,n}$ calculează

$$T := T + (f(x[i]) + f(x[i-1]))$$

$$U := U + f(y[i])$$

$$V := V + f(z[i])$$

6. Calculul aproximației integralei:

$$S = \left(\frac{(b-a)}{8 \cdot n}\right) \cdot (T + 3 \cdot U + 3 \cdot V)$$

7. Afișarea rezultatului: Tipărește S; STOP.

Exemple numerice:

1. Se consideră funcția $f(x) = \frac{1}{1+x}$. Să se calculeze valoarea aproximativă a integralei $\int_0^1 f(x)\,dx$, cu $n = 10$, folosind formula lui Newton, considerând $a = 0$, $b = 1$, și $f(x) = \frac{1}{1+x}$.
2. Să se utilizeze formula de cuadratură a lui Newton pentru n=20 la calculul integralei

$$\int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x}\,dx$$

cu $\epsilon = 0,016729$ și apoi să se calculeze o aproximație pentru lungimea orbitei Pământului în jurul Soarelui

$$L(P) = 4c \int_0^{\frac{\pi}{2}} \sqrt{1 - \epsilon^2 \sin^2 x}\,dx,$$

unde $c = 149,6 \times 10^6$ km. Se consideră datele de intrare $a = 0$, $b = \frac{\pi}{2}$,

$$f(x) = 4c \cdot \sqrt{1 - \epsilon^2 \sin^2 x}.$$

Să se compare rezultatele obținute folosind cuadratura trapezului, cuadratura perturbată a trapezului și cuadratura lui Simpson.

CODUL DE BAZĂ PENTRU PROGRAMUL DE LABORATOR ÎN C#

Clasa FormulaLuiSimpson moștenește clasa FormulaDeCuadratura și implementează formula lui Simpson pentru aproximarea integralelor definite. Constructorul clasei FormulaLuiSimpson primește intervalul de integrare, numărul de diviziuni și funcția, după care inițializează datele și execută calculele necesare.

Algorithm 38 Clasa FormulaLuiSimpson

```
public class FormulaLuiSimpson :  FormulaDeCuadratura
{
    public FormulaLuiSimpson(decimal a, decimal b, int n,
        Func<decimal, decimal> f) :  base(a, b, n, f)
    {
        AproximareIntegrala();
    }
    public static void Exemple()
    {
        // Exemple
    }
    protected override void AproximareIntegrala()
    {
        // Algoritm
    }
}
```

Pentru formula corectată de cuadratură a lui Simpson folosim clasa FormulaCorectataALuiSimpson ce moștenește clasa FormulaLuiSimpson și implementează formula corectată a lui Simpson pentru aproximarea integralelor definite. Constructorul clasei utilizează aceleași date ca și formula lui Simpson la care se adaugă valorile derivatei funcției în capetele intervalului. După inițializarea acestora, sunt executate calculele necesare.

Algorithm 39 Clasa FormulaCorectataALuiSimpson

```
1: public class FormulaCorectataALuiSimpson :  FormulaLuiSimpson
2: {
3:     protected decimal dfa, dfb;
4:     public FormulaCorectataALuiSimpson(decimal a, decimal b, int n,
5:         Func<decimal, decimal> f, decimal dfa, decimal dfb)
6:         :  base(a, b, n, f)
7:     {
8:         this.dfa = dfa;
9:         this.dfb = dfb;
10:        AproximareIntegrala();
11:    }
12:    public static void Exemple()
13:    {
14:        // Exemple
15:    }
16:    protected override void AproximareIntegrala()
17:    {
18:        // Algoritm
19:    }
20: }
```

În ceea ce privește formula de cuadratură a lui Newton, asemănător se creează clasa FormulaLuiNewton ce moștenește clasa FormulaDeCuadratura și implementează formula de cuadratură a lui Newton pentru aproximarea integralelor definite. Constructorul clasei primește aceleași date ca și celelalte formule de cuadratură. După inițializarea acestora, sunt executate calculele necesare.

Algorithm 40 Clasa FormulaLuiNewton

```
1: public class FormulaLuiNewton :  FormulaDeCuadratura
2: {
3:     public FormulaLuiNewton(decimal a, decimal b, int n,
4:         Func<decimal, decimal> f) :  base(a, b, n, f)
5:     {
6:         AproximareIntegrala();
7:     }
8:     public static void Exemple()
9:     {
10:        // Exemple
11:    }
12:    protected override void AproximareIntegrala()
13:    {
14:        // Algoritm
15:    }
16: }
```

Capitolul 15

Bibliografie

[1] J.H. Ahlberg, E.H. Nilson, J.L. Walsh,The Theory of Splines and Their Applications,.Academic Press, New York, London, 1967

[2] H. Akima, A new method for interpolation and smooth curve fitting based on local procedures, J.Assoc.Comput.Mach.4 (1970) 589–602

[3] P. Blaga, S. Pop, Gh. Coman, Analiza numerica : lucrari de laborator, Universitatea Babes-Bolyai Cluj-Napoca, 1994

[4] C. M. Bucur, C. A. Popeea, Gh. Gh. Simion, Matematici speciale. Calcul numeric, Ed. Didactica si Pedagogica, Bucuresti, 1983

[5] B. Demidovitch, I. Maron, Elements de calcul numerique, Ed. Mir, Moscow, 1979

[6] C. Iacob (coord.), Matematici clasice si moderne, vol.4, Ed. Tehnica, Bucuresti, 1983

[7] R. Plato, Concise Numerical Mathematics, in: Graduate Studies in Mathematics, vol. 57, AMS Providence, Rhode Island, 2003

[8] D.D. Stancu, Gh. Coman, O. Agratini, R. Trîmbitaş, Analiza numerica si teoria aproximarii, vol.1, Presa Univ. Clujeana, Cluj-Napoca, 2001

[9] D.D. Stancu, Gh. Coman, P. Blaga, Analiza numerica si teoria aproximarii, vol.2, Presa Univ.Clujeana, Cluj-Napoca, 2002

[10] O. Stanasila, Analiza matematica, Editura Didactica si Pedagogica Bucuresti 1981.

[11] J. Stoer, R. Bulirsch, Introduction to numerical analysis, Springer-Verlag, 1993